BCSE308P · Computer Networks Lab · Innovative Evaluation Practices

HOPTRACE

Network Path Fault Localization through Application-to-Hop Correlation

DELIVERABLE	**Assignment 8: Project Report** (PDF)
STUDENT / AUTHOR	**Krish Chirag Shah** (sole author)
REGISTRATION NO.	**24BCE0624**
COURSE	BCSE308P: Computer Networks Lab
SLOT	F2
TEAM NUMBER	24
SUBMISSION	VTOP · Assignments 8, 9 and 10 (report, presentation, GitHub repository)
DATE	27 October 2026
SOURCE CODE	GitHub repository **F2_Team24**: https://github.com/KrishUsesGit/F2_Team24

Research prototype. HOPTRACE estimates where application degradation *appears to begin* along an observed path. It does not establish which device caused it. The current detector has **not** been independently validated; the one independent evaluation (Phase 4) **missed** its false-attribution target (3/12 = 25% against at most 10%).

Report built from the project repository; every number is read from files in `evaluate/` and `experiments/`.

Contents

Abstract 3

1. Introduction and Problem Statement 4

2. Project Objectives and Scope 4

3. Existing Problem and Motivation 5

4. Proposed Solution and Design Rationale 5

- 4.1 Design decisions and why 6
- 4.2 What the result means, and what it does not 6
- 4.3 Phases of the project 7

5. System Architecture 8

- 5.1 Network testbed 8
- 5.2 Network path probing 8
- 5.3 Application telemetry 9
- 5.4 Correlation engine 9
- 5.5 Evidence and interpretation layer 9
- 5.6 Frontend dashboard 10
- 5.7 Fault injection and ground-truth isolation 10
- 5.8 Evaluation workflow 10

6. Technologies and Software Requirements 11

7. Implementation Details 13

- 7.1 Components and responsibilities 13
- 7.2 Data flow and file formats 13
- 7.3 Key source excerpts 14
- 7.4 How the dashboard uses stored results 15
- 7.5 How impairments are configured and evaluated 15

8. Network Topology and Experimental Setup 15

- 8.1 Topology 15
- 8.2 Routing and calibration 16
- 8.3 Probe collection, application RTT collection and fault injection 17
- 8.4 Trial sets 17
- 8.5 Topology verification (run for this report) 17

9. Algorithms and Methodology 19

- 9.1 Synchronisation into 500 ms buckets 19
- 9.2 Onset detection (median / MAD, sustained) 19
- 9.3 Timing alignment and downstream persistence 20
- 9.4 Evidence representation, confidence and interpretation 20
- 9.5 Limitations of the methodology 20
- 9.6 Algorithm selection (Phases 4 and 4b) 21

10. Dashboard and User Interaction 21

- 10.1 Trial selection 21
- 10.2 RTT visualisation: the symptom first 22
- 10.3 Path and hop inspection 23
- 10.4 Stored evidence, kept apart by kind 25
- 10.5 Guided explanation mode 26

11. Testing and Verification 28

- 11.1 Python unit tests: what they cover 29
- 11.2 Frontend tests: what they cover 30
- 11.3 Reproducible commands 30

12. Experimental Results and Analysis 32

- 12.1 Phase 4: detector mad on the held-out set (independent) 32
- 12.2 Phase 4b: detector mad_sustained on the same set (exposed) 33
- 12.3 Per-trial outcomes on the held-out set 33
- 12.4 Analysis 34
- 12.5 The recorded demo run (not an evaluation) 34

13. Limitations and Threats to Validity 35

14. Installation and Execution Guide 37

- 14.1 Environments 37
- 14.2 Get the code 37
- 14.3 Run the unit tests (safe, no root) 37
- 14.4 Build and verify the testbed (modifies the WSL system) 37
- 14.5 Run the live demo 38
- 14.6 View the dashboard 38
- 14.7 Selecting and running experiments 38
- 14.8 Troubleshooting 39
- 14.9 Cleanup 39

15. Conclusion and Future Work 39

- 15.1 Conclusion 39
- 15.2 Future work (recorded, not yet authorized) 40

16. References and Technical Documentation 40

17. Appendix 42

- A. Repository structure 42
- B. Important configuration files 42
- C. Test commands 42
- D. Additional dashboard screenshots 43
- E. How this report was produced 44

Abstract

When a networked application slows down, the application can observe *that* it is slow but not *where* on the network path the slowdown starts. Conventional tools split the problem in two: `ping` and `traceroute` measure the path without knowing what the application experienced, while application monitoring measures the application without knowing the path. HOPTRACE joins the two views. It probes every hop of a path and samples the application's own round-trip time (RTT) on **one shared clock**, detects when each series starts to behave abnormally, and reports the earliest hop whose onset lines up in time with the application's. Its output is deliberately bounded: *"Degradation appears to begin at or after Hop N."* It never says that a particular router caused anything.

The system is implemented end to end on a controlled testbed of six Linux network namespaces (a client, four routers and a server). It consists of a raw-socket path prober, a hand-written RFC 6455 WebSocket application client and server, a fault controller that injects `tc netem` / `tbf` impairments with ground truth isolated by operating-system permissions, a correlation engine (median/MAD onset detection, timing alignment, typed evidence), an interpretation layer, and a React + TypeScript dashboard that displays stored results. All Python code uses only the standard library.

Evaluation was honest about failure. In Phase 4, the only independent held-out run, the selected detector (`mad`) **missed the false-attribution target: 3/12 = 25% against at most 10%**. A revised detector (`mad_sustained` , Phase 4b) scored 1/12 = 8.3%, but on the **same held-out set after its traces had been inspected**, so that figure is a historical record and **not** independent validation. Loss and bandwidth faults were not localized (0 of 4 on the exposed held-out set and 0 of 4 on the Phase 4b tuning set). Confidence is a fixed heuristic, not a probability.

Report at a glance: what was verified when this report was produced (27 October 2026)

- **117 Python unit tests** pass (WSL2 Ubuntu 24.04, Python 3.12.3) and the repository hygiene check passes.
- **310 frontend tests** in 14 files pass; TypeScript strict typecheck, ESLint with zero warnings, production build and the distribution-safety check pass; 40 real-browser layout checks and a four-trial browser session pass (Windows 11, Node 24.14.1).
- The **six-namespace testbed** was built, verified (200/200 packets, 0% loss; five hops in order, twice) and torn down in WSL2.
- **Not re-run now:** the full live demo (it overwrites a recorded trial), the historical evaluation scripts (they overwrite result files), an offline test of the dashboard, and a clean-clone test. Section 11 lists these explicitly.

1. Introduction and Problem Statement

A user reports that an application feels slow. The operator can look at the application's response times, and can run `traceroute` to list the routers along the way. What neither tool answers is the question that matters during diagnosis: **at which point along the path did the application's degradation begin to appear?**

Answering this requires the application's behaviour and the path's behaviour to be compared *in time*. If the application's RTT jumps at 25.0 s and the probe RTT to Hop 3 jumps at 25.0 s while Hops 1 and 2 stay flat, the evidence points at the segment between Hop 2 and Hop 3. If the two series use unsynchronised clocks, or if the probes are sent from a different place from where the application runs, that comparison is unreliable.

Problem statement. Given an application whose round-trip time degrades while a path of known hops is observed, estimate the earliest hop at which the degradation appears to begin, state the evidence and its limits, and refuse to claim a cause.

1.1 What the project is, and is not

HOPTRACE is	HOPTRACE is not
A controlled-testbed research prototype for studying application-to-hop timing correlation.	A production monitoring product or dashboard for live networks.
A tool that estimates *where along an observed path* degradation appears to begin.	A root-cause analyser. It does not say which router, link or configuration caused anything.
A system that reports unknown and missing observations as unknown.	A system that treats silence as health. "No onset detected" is not "healthy".
An honestly evaluated prototype, including its failures.	An independently validated detector. No such validation exists yet.

2. Project Objectives and Scope

2.1 Objectives

1. Build a reproducible testbed in which the path has a known number of hops and faults can be injected on a chosen link.
2. Measure the path hop by hop *and* the application's own RTT against a single shared clock.
3. Detect, for every series, when it starts behaving abnormally, and align those onsets in time.
4. Report the result in evidence-bounded language, keeping measured facts, inferences, correlations, confidence and unknowns visibly separate.
5. Keep the record of what was injected (ground truth) out of reach of the localization engine, and score the engine only afterwards.
6. Present stored results in an interactive dashboard that does not recompute or embellish them.
7. Evaluate candidate detectors on separate tuning and held-out trial sets, and disclose the outcome, including misses.

2.2 In scope and out of scope

In scope (implemented)	Out of scope (not implemented, not claimed)
Linear six-node namespace testbed (client, 4 routers, server)	Real or multi-path networks; ECMP; IPv6
Raw-socket TTL probing (TCP-SYN default, UDP optional), 5 hops	Per-hop packet-level capture or per-packet analysis
Application ping RTT over a hand-written WebSocket	Real applications; other protocols
Latency, jitter, loss and bandwidth impairments via `tc`	Reliable localization of loss and bandwidth faults (unproven)
Median/MAD onset detection, timing alignment, typed evidence	Calibrated confidence; causal inference
Stored-result dashboard with guided explanation	Live streaming of new measurements into the dashboard
Phase 4 / 4b evaluation on small trial sets	The sealed, full-scale evaluation (spec Phase 5) was not run

3. Existing Problem and Motivation

`ping` and `traceroute` report per-hop reachability and RTT. They describe the path, but say nothing about what a particular application sees. Application performance monitoring has the opposite blind spot: it sees the symptom but not the path. A network engineer therefore correlates the two by hand, which is slow and hard to reproduce.

Two practical difficulties motivate HOPTRACE's design:

- **Clock alignment.** To say that two onsets coincide, both measurements must share a timeline. HOPTRACE runs the prober and the application client in the same network namespace and reads the same `CLOCK_MONOTONIC` clock, so their timestamps are directly comparable without any clock synchronization protocol.
- **Honest uncertainty.** Timing coincidence is not causation, and a hop that does not answer probes is not necessarily healthy. A tool that quietly turns missing data into a confident verdict is worse than no tool. HOPTRACE therefore keeps "unknown" as a first-class state and restricts its sentences to a fixed, evidence-bounded template.

4. Proposed Solution and Design Rationale

HOPTRACE runs a four-step loop: **measure** → **detect** → **align** → **report**.

1. **Measure.** A raw-socket prober sends TTL-limited probes to hops 1–5 every 500 ms; an application client pings its server at 20 Hz. Both write timestamped JSON-lines files using one monotonic clock.
2. **Detect.** Both streams are aggregated into 500 ms buckets. For each series the detector computes a median/MAD baseline and finds the first point where the series stays above *median + 5·MAD* for three consecutive buckets.
3. **Align.** Starting from the shallowest hop, find the first hop whose onset is within 1.5 s of the application's onset *and* whose deeper hops also show concurrent onsets (persistence downstream).
4. **Report.** Convert the result into a typed evidence record and a fixed-template sentence: *"Degradation appears to begin at or after Hop N."*

4.1 Design decisions and why

Decision	Rationale	Where in the code
Prober and application client co-located; one `CLOCK_MONOTONIC`	Makes onsets directly comparable with no cross-host clock error.	`prober/prober.py`, `app/client.py`
Fault is applied to a router's *egress* link (the link leaving Hop N)	Hop N's own reply is generated before that link, so the first hop that can show the fault is N+1. The evaluation uses this "Hop N onward" rule.	`faultctl/controller.py`, `evaluate/metrics.py`
Ground truth stored under a separate OS identity, outside the repository	The engine cannot read what was injected: isolation by permissions *and* by construction (`correlate/` never imports the ground-truth module).	`faultctl/groundtruth.py`, `setup_identities.sh`
Detectors compared on tuning data, scored on held-out data	Avoids choosing an algorithm by assumption; exposes over-claiming.	`evaluate/`
Unknown hops stay unknown	A silent hop is not evidence of health; it lowers confidence instead.	`correlate/pipeline.py`, `evidence.py`
Fixed result templates with forbidden-phrase assertions	Prevents causal language ("caused", "is faulty") from ever being produced.	`present/interpret.py`, `path_bundle.py`
Dashboard displays stored bundles only	The browser cannot silently change a detector result.	`frontend/`

4.2 What the result means, and what it does not

The single most important design point is the **boundary of the claim**. HOPTRACE's output has exactly four result states, each with a locked sentence:

Result state	When	Locked sentence (as in `present/interpret.py`)
Localized	Application onset found, hop range found, confidence *high*	"Degradation appears to begin at or after Hop N."
Partially localized	Hop range found, confidence *medium* or *low*	"We detected degradation, but the exact location is uncertain."
Not localized	Application onset found, no corroborated hop	"We detected application degradation, but couldn't establish a reliable network-path location."
No degradation	No application onset detected	"No significant degradation was detected during this test."

Reading the result correctly

- The result says **when changes were observed along the path**, not **why**. It does not name a faulty device.
- **Timing correlation does not establish causation.** A hop whose onset lines up with the application's is timing evidence only.
- **"No significant degradation was detected" means no onset was detected.** It can also occur for a real fault that covers most of the observation window, or when there is too little data (Section 9.5).
- **Unknown ≠ healthy** and **confidence ≠ probability** (Section 9.4).

4.3 Phases of the project

The project follows a locked specification (`hoptrace-phase-0-5-spec.html`) whose phase numbering is authoritative. Several labels inside code comments and commit messages predate that numbering; the evidence ledger (`docs/EVIDENCE_LEDGER.md`) records the mapping.

Phase	Content	Where	State
0	Controlled testbed: 6 network namespaces	`testbed/phase0/`	Implemented; re-verified for this report
1	Hop-level probing (raw sockets)	`prober/`	Implemented
2	Application telemetry (WebSocket)	`app/`	Implemented
3	Fault controller and isolated ground truth	`faultctl/`	Implemented
4	Correlation engine and algorithm selection	`correlate/` , `evaluate/`	Engine implemented; **independent gate failed**
4b	Diagnostic retuning after the Phase 4 failure (not a spec phase)	`evaluate/*_4b.py`	Scored on an **exposed** set
5	Sealed evaluation harness and calibration check	-	**Not started**
6	UI and demo	`frontend/` , `ui/` , `demo/` , `present/`	Built early; the spec would have required Phase 5 first

The "Phase 5" label inside `present/` and in commit `c202be0` refers to the *result interpretability layer*, not to spec Phase 5. Phase 6 was built before the Phase 5 evaluation, which the specification would not have allowed; this is disclosed, not hidden.

5. System Architecture

HOPTRACE has two halves that are kept apart on purpose. The **observation and analysis path** measures, detects, aligns and presents. The **experiment control and evaluation** half injects faults, records what was injected, and scores results afterwards. Information flows from the second half to the first *only* through the physical network (the impairment itself), never as labels.

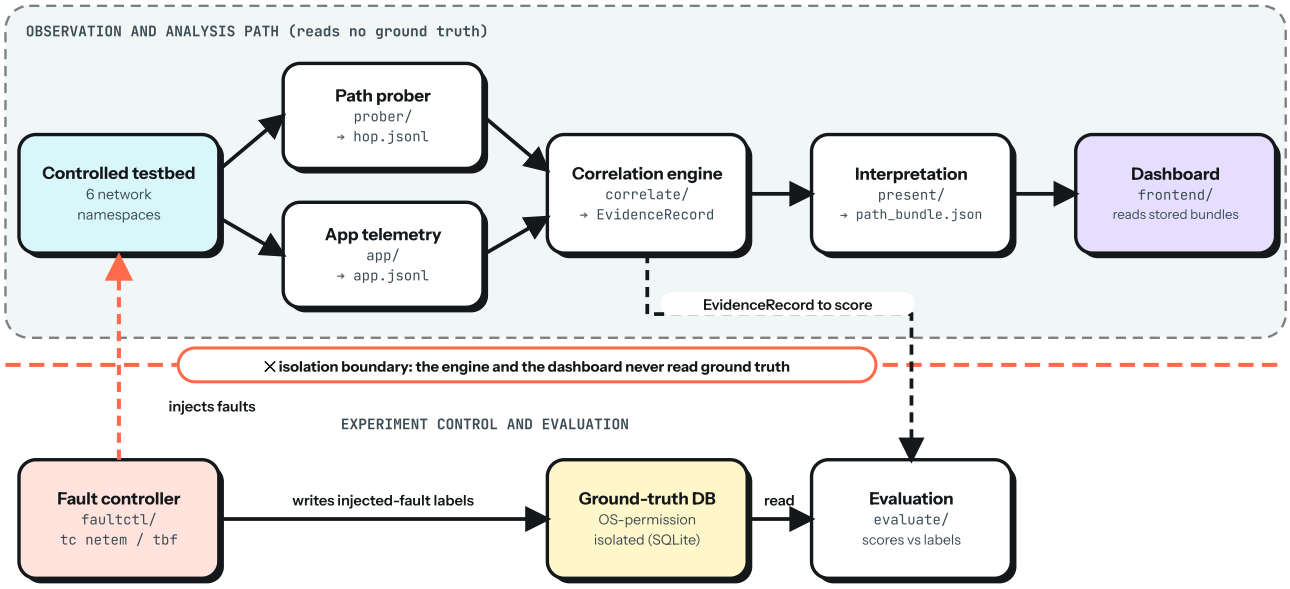


Figure 1. HOPTRACE architecture. Solid arrows are data flow; coral dashed arrows are fault injection. The dashed horizontal line is the isolation boundary: nothing above it can read ground truth.

5.1 Network testbed

Six Linux network namespaces form a linear path: `ns-client`, `ns-r1` … `ns-r4`, `ns-server`, joined by five veth pairs on five `/30` subnets (`10.10.N.0/30`). IP forwarding is enabled only in the four router namespaces; static routes are explicit per subnet. The probe path therefore has exactly five measurable hops: four routers and the server as Hop 5. Details are in Section 8.

5.2 Network path probing

`prober/prober.py` runs inside `ns-client` as root (raw sockets). In each *round* it sends two probes per TTL for TTL 1…5, sequentially (one outstanding probe at a time, one fixed 5-tuple per round), with a 0.3 s timeout; rounds start every 0.5 s. Two transports share one interface:

- **TCP-SYN** (default in the demo and trials): a hand-built IP+TCP header with `IP_HDRINCL`, destination port 80; a router answers with ICMP Time-Exceeded, the server with a TCP reply.
- **UDP**: destination port `33434 + round_id`; routers answer with ICMP Time-Exceeded, the server with ICMP Destination-Unreachable.

Every probe becomes one `HopSample` row. A probe with no answer is recorded as `responded: false` with null IP and RTT; it is never interpolated.

5.3 Application telemetry

`app/client.py` opens one persistent WebSocket/TCP connection (the protocol is hand-implemented in `app/ws_protocol.py` following RFC 6455) from `ns-client` to `app/server.py` in `ns-server`. It sends a **20 Hz ping stream** and a Poisson-arrival **interaction stream** (mean 2 events/s, 150 ms budget). The server only echoes sequence numbers; RTT is computed entirely client-side from the same `CLOCK_MONOTONIC` calls the prober uses. Three states are kept distinct: *within budget*, *late* (measured RTT above 150 ms, interactions only) and *missing* (no reply within 2 s; RTT stays null and is never recorded as a large number). Only the ping stream feeds the detector.

5.4 Correlation engine

`correlate/` turns the two raw files into a typed result in four small modules (Figure 2): `synchronize.py` (common 500 ms timeline), `onset_candidates_v2.py` (the frozen `mad_sustained` detector), `align.py` (cross-series alignment and downstream persistence) and `evidence.py` (typed claims). `pipeline.py` orchestrates them. The engine reads only the two files it is given; there is no code path that could read what was injected.

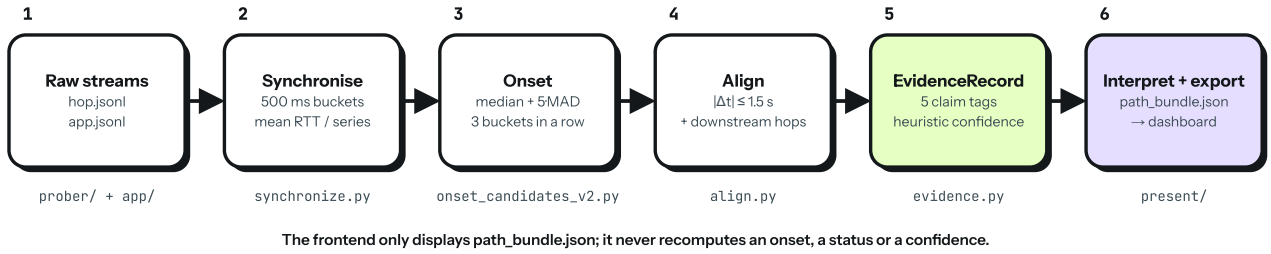


Figure 2. The analysis data flow, from raw measurements to the stored bundle that the dashboard displays.

5.5 Evidence and interpretation layer

The correlation engine's output is an `EvidenceRecord`: a list of claims, each carrying exactly one of five tags. `present/interpret.py` maps the record to one of four locked result sentences, `present/technical_view.py` exposes the underlying numbers, and `present/path_bundle.py` assembles the UI bundle. These modules perform no detection, alignment or confidence computation; they re-express stored fields and assert that no causal phrase ("caused", "is faulty", "is broken", "responsible for") is ever produced.

Tag	Meaning	Example (from `demo-run-01`)
`fact`	An observation from the data. No interpretation.	"Hop 3 RTT onset detected at t=41726703720 (method=mad_sustained_v1)."
`inference`	What the facts are consistent with. Not a measurement.	"Evidence is consistent with degradation first appearing at or after Hop 3, persisting through Hop 5."
`correlation`	A timing association; carries the lag in ms.	"...temporally correlated with the Hop 3 path-level onset (this is an association, not a causal claim)."
`confidence`	A fixed heuristic score and level.	level *high*, score 1.0
`unknown`	Observations that are missing. Never assumed healthy.	"No hops were UNKNOWN for this trial."

5.6 Frontend dashboard

`frontend/` is a React + TypeScript (strict) + Vite + Tailwind + Motion application. It has no backend. A small Vite plugin serves an explicit allowlist of stored files (the four `path_bundle.json` files and two aggregate evaluation projections) in development and emits the same files into `dist/data/` for the production build. The page shows the application's RTT chart, a three-dimensional client-to-server path with selectable hops, a per-hop inspector, the boundary statement, the evidence grouped by kind, the stored confidence and its limits, the stored evaluations, the limitations, and an optional guided explanation. The browser **formats and displays** stored values; it never recomputes an onset, a status or a confidence. Section 10 walks through it with screenshots.

5.7 Fault injection and ground-truth isolation

`faultctl/controller.py` runs as the OS user `hoptrace-truth`. It may reach `tc` inside the four router namespaces *only*, through a narrow sudoers rule created by `faultctl/setup_identities.sh`. Each injection records an event (trial, hop, type, magnitude, start/end time on the monotonic clock, the exact `tc` command) in a SQLite database at `/var/lib/hoptrace/groundtruth.db`, which is **deliberately outside the repository** on the native Linux filesystem (mode 0640, directory 0750). The analysis identity `hoptrace-engine` is not in that group and receives *Permission denied*; the evaluation identity `hoptrace-eval` is in the group and may read it. `demo/run_demo.sh` checks this live before injecting anything.

Impairment	Applied as (`tc`, on the router egress interface)	Parameters recorded
latency	`netem delay D ms`	delay_ms
jitter	`netem delay D ms J ms`	delay_ms, jitter_ms
loss	`netem loss P %`	loss_pct
bandwidth	`tbf rate R kbit burst 32kbit latency 400ms`	rate_kbit (burst and latency are fixed defaults)

A fault at "hop N" is applied to the egress interface of `ns-rN`, i.e. the link *leaving* Hop N: Hop 1 → `ns-r1:veth-s2-a`, Hop 2 → `ns-r2:veth-s3-a`, Hop 3 → `ns-r3:veth-s4-a`, Hop 4 → `ns-r4:veth-s5-a`. Hop 5 (the server) is not a target. Consequently the first hop whose measurement can show the fault is **N+1**, and the evaluation scores against `min(N+1, 5)`.

What the isolation does and does not guarantee

- **Does:** an unprivileged analysis process cannot read the database (checked live by the demo script); `correlate/` never imports the ground-truth module.
- **Does not:** root can read the database; trial directory names for historical trials encode the fault type and hop; the evaluation result files in `evaluate/` contain labels and are readable by any local process. It is a permission test, not a complete security audit.

5.8 Evaluation workflow

1. Record trials (`hop.jsonl`, `app.jsonl`, a minimal `trial_meta.json`) while the fault controller injects (or does not inject) an impairment.
2. Run candidate detectors on a **tuning** set; select one by a fixed rule (primary: within-one-hop accuracy; tie-breaks: false attribution, then Brier score) and write a frozen configuration file.
3. Score the frozen configuration once on a **held-out** set as `hoptrace-eval`, which is the only place engine output is joined to ground truth.
4. Report the metrics, including a Wilson 95% interval, and any failure.

The metrics are defined in `evaluate/metrics.py`. A **false attribution** is a confident (*medium* or *high*) claim that is either wrong or made on a clean control. The target for the project is at most 10%.

6. Technologies and Software Requirements

Only what the repository actually uses is listed. **Required** items are needed to run HOPTRACE or its dashboard; **optional** items are development, verification or documentation tools.

Table 6.1 Required: measurement, analysis and testbed (Linux)

Component	Version used / verified	Role
WSL2 Ubuntu 24.04	kernel 6.18.33.2-microsoft-standard-WSL2	The only environment on which the testbed was run (README: supported and tested platform)
Python	3.12.3	All measurement, analysis and evaluation code. **Standard library only, with no third-party Python packages**
iproute2 (`ip` , `tc`)	6.1.0 (libbpf 1.3.0)	Namespaces, veth pairs, `netem` / `tbf`
`traceroute`	as installed in the WSL distribution	Used only by `verify_topology.sh`
root, `sudo` , bash	-	Raw sockets, namespaces and `tc` require root; identities use a restricted sudoers rule
SQLite (stdlib `sqlite3`)	bundled with Python	Ground-truth event store

Table 6.2 Required to run the dashboard (declared in `frontend/package.json` ; exact resolved versions are in `package-lock.json`)

Package	Declared range	Role
Node.js / npm	`node ≥ 22.12` (verified on 24.14.1 / npm 11.11.0)	Runtime for build and preview
react, react-dom	^19.3.0	UI framework
typescript	~6.0.3 (dev)	Strict type checking
vite, @vitejs/plugin-react	^8.3.4, ^6.1.2 (dev)	Bundler / dev and preview server
tailwindcss, @tailwindcss/vite	^4.3.3 (dev)	Styling
motion	^14.2.0	Animation (removed under *prefers-reduced-motion*)
zod	^4.6.7	Runtime validation of stored bundles
@fontsource-variable/*	^5.3.0	Self-hosted fonts (Unbounded, Instrument Sans, JetBrains Mono); no CDN at runtime

Table 6.3 Optional: verification, development and documentation

Tool	Used for
vitest, @testing-library/*, jsdom, axe-core	Frontend unit, component, accessibility and contrast tests (`npm test`)
eslint 9, typescript-eslint, eslint-plugin-jsx-a11y	Lint with zero warnings allowed (`npm run lint`)
Chrome or Edge	`npm run check:layout` and `npm run check:preview` (real-browser checks)
matplotlib, python-pptx, PyMuPDF, a Chromium browser	Only to regenerate the figures, this report and the presentation (`submission/build/`). Not needed to run HOPTRACE

The older static page `ui/` loads GSAP and Google Fonts from CDNs and is kept for history only; the submitted dashboard is `frontend/`. The environment used for *development* of this report was Windows 11 with WSL2; the environment *required* to run the testbed is WSL2 Ubuntu 24.04 (or an equivalent Linux with network namespaces and root). Windows-native Python is not supported.

7. Implementation Details

7.1 Components and responsibilities

Directory	Main files	Responsibility
`testbed/phase0/`	`setup_topology.sh`, `verify_topology.sh`, `teardown_topology.sh`, `lib/common.sh`	Build, check and remove the six-namespace topology; single source of names and addresses
`prober/`	`prober.py`, `raw_probe.py`, `hopsample.py`	TTL-limited raw-socket probes; `HopSample` records
`app/`	`client.py`, `server.py`, `ws_protocol.py`, `apprttsample.py`	WebSocket ping/interaction telemetry; `AppRttSample` records
`faultctl/`	`controller.py`, `groundtruth.py`, `setup_identities.sh`	Inject/clear `tc` impairments; ground-truth store; OS identities
`correlate/`	`synchronize.py`, `onset*.py`, `align.py`, `evidence.py`, `pipeline.py`	Timeline, onset detection, alignment, typed evidence
`present/`	`interpret.py`, `technical_view.py`, `path_bundle.py`, `report.py`	Locked sentences; technical view; UI bundle; text report
`evaluate/`	`metrics.py`, `*_analysis*.py`, `held_out_*.py`, frozen configs and results	Metrics, algorithm selection, held-out scoring (historical record)
`demo/`	`run_demo.sh`, `demo_script.md`	Real end-to-end run on the live testbed
`frontend/`	`src/`, `vite/`, `scripts/`	Dashboard, data allowlist, browser checks
`experiments/`	40 trial directories	Raw evidence: `hop.jsonl`, `app.jsonl`, `trial_meta.json`, and (for some) `evidence*.json` and `path_bundle.json`
`tools/`, `docs/`	`check_repo_hygiene.py`, `EVIDENCE_LEDGER.md`	Read-only repository check; what each data set may be used for

7.2 Data flow and file formats

There is no database between stages; each stage reads and writes plain files. Times are nanoseconds on `CLOCK_MONOTONIC`.

HOP.JSONL: ONE LINE PER PROBE (`HOPSAMPLE`)

```
{"round_id":0,"ttl":1,"hop_ip":"10.10.1.2","rtt_ms":0.050222,"responded":true,"probe_index":0,
 "t_send_mono_ns":16726703720,"t_recv_mono_ns":16726784215}
```

APP.JSONL: ONE LINE PER APPLICATION SAMPLE (`APPRTTSAMPLE`)

```
{"seq":0,"kind":"ping","t_send_mono_ns":16793313189,"t_recv_mono_ns":16793623395,
 "rtt_ms":0.310206,"responded":true,"missed_budget":false}
```

EVIDENCERECORD (`EVIDENCE*.JSON`)

```
{ "trial_id": "...", "generated_at_mono_ns": ..., "claims": [
    {"tag":"fact", "text":"...", "data_ref":{...}},
    {"tag":"inference", "hop_range":{"min":3,"max":5}, "text":"..."},
    {"tag":"correlation", "lag_ms":0.0, "text":"..."},
    {"tag":"confidence", "level":"high", "score":1.0, "factors":{...}},
    {"tag":"unknown", "hops":[], "text":"..."} ] }
```

PATH_BUNDLE.JSON (THE DASHBOARD'S DATA)

Top-level keys: `trial_id` , `plain_language` (`result_state` , `summary`), `hop_range` , `confidence` , `alignment_delta_ms` , `observation_window` , `evidence_record_claims` , `app` , `hops` , `unknown_hops` , `evaluation` . Each hop entry carries `ttl` , `ip` , `status` (*degraded* = corroborated onset, *healthy* = no corroborated onset, *unknown*), baseline/observed mean RTT, `onset_ns` , `sample_count` and templated `why_highlighted` text. A hop's status is a pure re-expression of existing evidence fields (`_hop_status` in `present/path_bundle.py`). There is no JSON Schema file or version field yet; the frontend validates bundles at load time with a Zod schema.

GROUND-TRUTH SCHEMA (`FAULTCTL/GROUNDTRUTH.PY`)

```
CREATE TABLE ground_truth_events (
    event_id TEXT PRIMARY KEY, trial_id TEXT NOT NULL, hop_target INTEGER NOT NULL,
    impairment_type TEXT NOT NULL, magnitude TEXT NOT NULL,
    t_start_mono_ns INTEGER NOT NULL, t_end_mono_ns INTEGER, tc_command TEXT NOT NULL );
```

The database is created on first use inside the running testbed environment; it is never committed (`*.db` is in `.gitignore`).

7.3 Key source excerpts

Sustained onset detection (`correlate/onset_candidates_v2.py` , the frozen Phase 4b detector; unchanged):

```
def detect_onset_mad_sustained(series, t, k_mad: float = 5.0, min_run: int = 3):
    values, times = _valid(series, t)
    if len(values) < MIN_VALID_POINTS:            # 5
        return None
    med = st.median(values)
    mad = st.median([abs(v - med) for v in values]) or 1e-9
    threshold = med + k_mad * mad
    run_start = None; run_len = 0
    for i, v in enumerate(values):
        if v > threshold:
            if run_start is None: run_start = i
            run_len += 1
            if run_len ⩾ min_run:                 # onset = first point of the run
                return OnsetResult(t_onset_ns=times[run_start],
                                   confidence_raw=(v - med) / mad, method=METHOD_MAD_SUSTAINED)
        else:
            run_start = None; run_len = 0
    return None
```

Alignment with downstream persistence (`correlate/align.py` , abridged):

```
DELTA_ALIGN_NS = 1_500_000_000          # 1.5 s: documented default, still unvalidated

for candidate in sorted(hop_onsets.keys()):
    onset = hop_onsets.get(candidate)
    if onset is None or not _concurrent(onset.t_onset_ns, app_onset.t_onset_ns):
        continue
    # every hop from `candidate` to the deepest measured hop must also show a concurrent onset
    ...
    if not all_persist:
        continue                            # isolated spike: keep looking deeper
    return LocalizationResult(hop_range={"min": candidate, "max": max(corroborating)}, ...)
```

Fault injection (`faultctl/controller.py`, abridged): the controller refuses any hop outside the four authorized targets and refuses to inject if the interface does not exist.

```
AUTHORIZED_TARGETS = {1: ("ns-r1", "veth-s2-a"), 2: ("ns-r2", "veth-s3-a"),
                      3: ("ns-r3", "veth-s4-a"), 4: ("ns-r4", "veth-s5-a")}
...
tc_args = ["qdisc", "add", "dev", iface, "root", "netem", "delay", f"{delay_ms}ms"]   # latency
```

7.4 How the dashboard uses stored results

- **Data source.** `vite/hoptraceData.ts` scans `experiments/*/path_bundle.json` and serves `data/index.json`, `data/bundles/<id>.json` and two aggregate projections of the evaluation files. Nothing else is reachable: raw traces (`*.jsonl`), the ground-truth database and arbitrary paths return no data (probed by about forty URLs in `vite/devServer.test.ts`).
- **Validation.** Each bundle is checked against a Zod schema at load time; a malformed bundle shows an error state rather than a guess.
- **No recomputation.** The browser formats stored values. Statuses, onsets, confidence and hop range come from the bundle.
- **Local-path hygiene.** The historical demo bundle records an absolute machine path; the file on disk is left unchanged (it is evidence), the served copy keeps only the base name, and `npm run verify:dist` fails on any local path in `dist/`.
- **Illustration vs measurement.** Moving packets in the 3-D scene are **illustrative only**; the bundles contain no packet-level observations, and the page says so next to the animation.

7.5 How impairments are configured and evaluated

The impairment type, hop and magnitude are chosen per trial and passed to `faultctl/controller.py inject`; `clear-latest` removes the open event by trial and hop. The historical trial sets used magnitude classes (*mild*, *moderate*, *severe*) and some *transient* faults that cover only part of the window; these are encoded in the trial directory names, not in a machine-readable file. For scoring, `evaluate/metrics.py` reads the ground-truth hop and takes `expected_hop = min(hop_target + 1, 5)`.

8. Network Topology and Experimental Setup

8.1 Topology

The testbed is a straight line of six namespaces. The client hosts both measurement processes; the server hosts the application endpoint; the four routers forward between them and are the only places a fault can be injected.

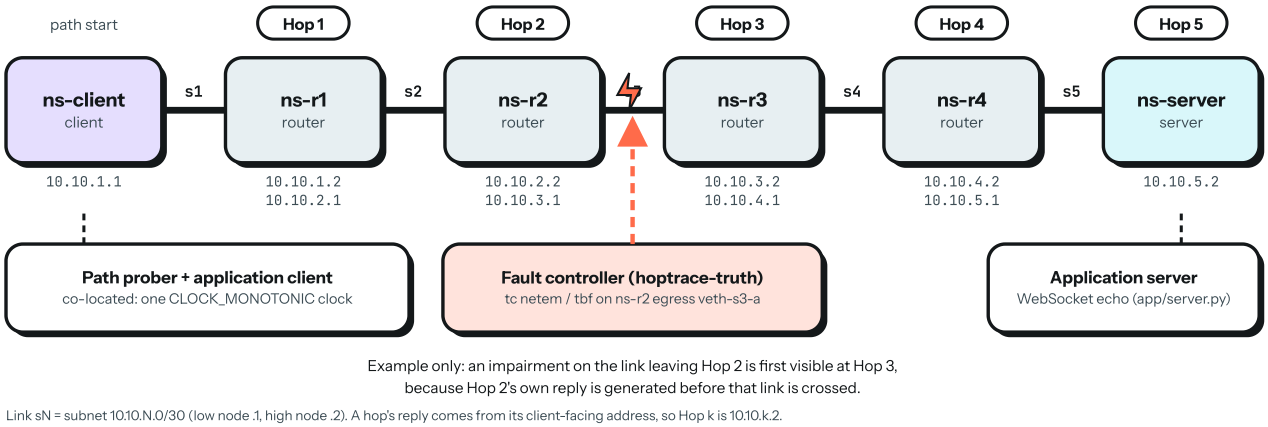


Figure 3. The six-namespace testbed as built by `testbed/phase0/setup_topology.sh`. Hops 1-4 are the routers; Hop 5 is the server itself. Addresses are those assigned by `testbed/phase0/lib/common.sh`.

Table 8.1 Addressing (subnet N is `10.10.N.0/30`; the lower node takes `.1`, the higher node `.2`)

Subnet	Low-side node : interface	High-side node : interface	Low addr	High addr
s1	ns-client : veth-s1-a	ns-r1 : veth-s1-b	10.10.1.1	10.10.1.2
s2	ns-r1 : veth-s2-a	ns-r2 : veth-s2-b	10.10.2.1	10.10.2.2
s3	ns-r2 : veth-s3-a	ns-r3 : veth-s3-b	10.10.3.1	10.10.3.2
s4	ns-r3 : veth-s4-a	ns-r4 : veth-s4-b	10.10.4.1	10.10.4.2
s5	ns-r4 : veth-s5-a	ns-server : veth-s5-b	10.10.5.1	10.10.5.2

The client address is `10.10.1.1` and the server address `10.10.5.2`. A router's probe reply is sourced from its client-facing address, so the five hops appear as `10.10.1.2, 10.10.2.2, 10.10.3.2, 10.10.4.2, 10.10.5.2`, which is exactly what the topology check confirms.

8.2 Routing and calibration

- **Forwarding and routes.** `net.ipv4.ip_forward=1` only in the four router namespaces. Static, explicit per-subnet routes (no aggregation); the client and server each have a single `10.10.0.0/16` route towards the path.
- **ICMP rate-limit calibration.** Linux's default `icmp_ratelimit` allows about one ICMP error per second per namespace, which silently suppressed most hop replies at a 500 ms probe cadence (about 60% matched in an early run). The setup script sets `icmp_ratelimit=0` *inside the four router namespaces and the server namespace only*. It does not disable packet loss or alter `netem` / `tbf` behaviour, and it does not touch the host's own namespace.

8.3 Probe collection, application RTT collection and fault injection

Table 8.2 Measurement parameters (locked in the specification unless noted)

Parameter	Value	Source
Hops measured	5 (TTL 1-5)	`prober.py: TTL_RANGE`
Probes per hop per round	2	`SAMPLES_PER_HOP`
Probe round interval	0.5 s	`ROUND_INTERVAL_S`
Probe timeout	0.3 s	`PROBE_TIMEOUT_S`
Application ping rate	20 Hz (50 ms)	`client.py: PING_INTERVAL_S`
Interaction arrivals	Poisson, mean 2/s	`INTERACTION_MEAN_RATE`
Interaction budget	150 ms	`INTERACTION_BUDGET_MS`
Missing-reply timeout	2.0 s	`MISSING_TIMEOUT_S` (a documented choice, not spec-locked)
Analysis bucket width	500 ms	`synchronize.py: BUCKET_WIDTH_NS`

Timeline of one trial. The app server starts in `ns-server`; the prober and the application client start together in `ns-client`; a fault, if any, is injected partway through; both measurement processes run to completion; the fault is cleared. All timestamps come from the same monotonic clock, so a fault's start time (in the ground-truth store) and an onset time (in the analysis) are on one scale, but only the evaluation, never the engine, can see the former.

8.4 Trial sets

Table 8.3 Experiment sets in `experiments/` (status as recorded in `docs/EVIDENCE_LEDGER.md`)

Set	Trials	Role	Status
`p3-*`	5	Exploratory runs of the first (provisional) pipeline	Exploratory; no reported accuracy
`p4-tune-*`	12	Tuning for Phase 4 (`mad` selected)	Tuning, historical
`p4-held-*`	12	Held-out for Phase 4	**Independent at Phase 4; EXPOSED from Phase 4b onward**
`p4b-tune-*`	10	Tuning for Phase 4b (`mad_sustained` selected)	Tuning, historical
`demo-run-01`	1	Live demonstration run	Demo; not an evaluation trial

Each 12-trial set contains four clean controls and eight fault trials (latency, jitter, loss and bandwidth, at different hops and magnitudes, some transient), plus trials in which one or two hops return no responses ("unknown" trials). The repository's evidence ledger notes that the 12-trial sets are far below the 120-trial tuning set and 288-cell matrix described in the specification; the reduction is disclosed in `evaluate/power_calc.py`.

8.5 Topology verification (run for this report)

On 27 October 2026 the topology was built, checked and torn down in WSL2 Ubuntu 24.04 with the project's own scripts (no ground-truth database, identity or recorded trial was touched):

```
bash testbed/phase0/setup_topology.sh      # → "topology up."  icmp_ratelimit calibration: 0 (ns-r1, expect
0)
bash testbed/phase0/verify_topology.sh
  Test 1: 200 packets transmitted, 200 received, 0% packet loss   rtt min/avg/max/mdev =
0.017/0.049/0.158/0.020 ms   PASS
  Test 2: run 1: 10.10.1.2 10.10.2.2 10.10.3.2 10.10.4.2 10.10.5.2
PASS
          run 2: 10.10.1.2 10.10.2.2 10.10.3.2 10.10.4.2 10.10.5.2
PASS
bash testbed/phase0/teardown_topology.sh   # → namespaces removed; `ip netns list` empty afterwards
```

The recorded demo trial (`demo-run-01`) was produced by an earlier live run of `demo/run_demo.sh` and was **not** regenerated for this report, because that script overwrites the recorded trial.

9. Algorithms and Methodology

9.1 Synchronisation into 500 ms buckets

Because the prober and the application client are separate processes with separate counters, there are no shared sequence IDs. Alignment instead uses the shared clock: all `t_send_mono_ns` values are placed on one timeline starting at the earliest sample, and divided into 500 ms buckets (the coarser of the two cadences; pings run at 20 Hz, probe rounds at 2 Hz). For bucket b, the application value is the **mean RTT of the pings that responded** in it, and the value for hop k is the mean RTT of the probes to TTL k that responded. A bucket with samples but no response is **unknown**; a bucket with no value is skipped by the detector and never interpolated.

9.2 Onset detection (median / MAD, sustained)

For one series with valid values $x_1 \ldots x_n$ ($n \geq 5$, otherwise no onset can be claimed):

$$m = \text{median}(x), \quad MAD = \text{median}(|x_i - m|), \quad T = m + k \cdot MAD \quad \text{with } k = 5 \ (\texttt{k_mad})$$

The **onset** is the time of the first bucket of the first run of at least `min_run` = 3 **consecutive** values above T. A single noisy bucket above T does not count. (If the series is perfectly flat the MAD is replaced by 10-9 to avoid division by zero.) These are exactly the operations in `detect_onset_mad_sustained` (Section 7.3). Figure 4 shows them on the recorded `demo-run-01`.

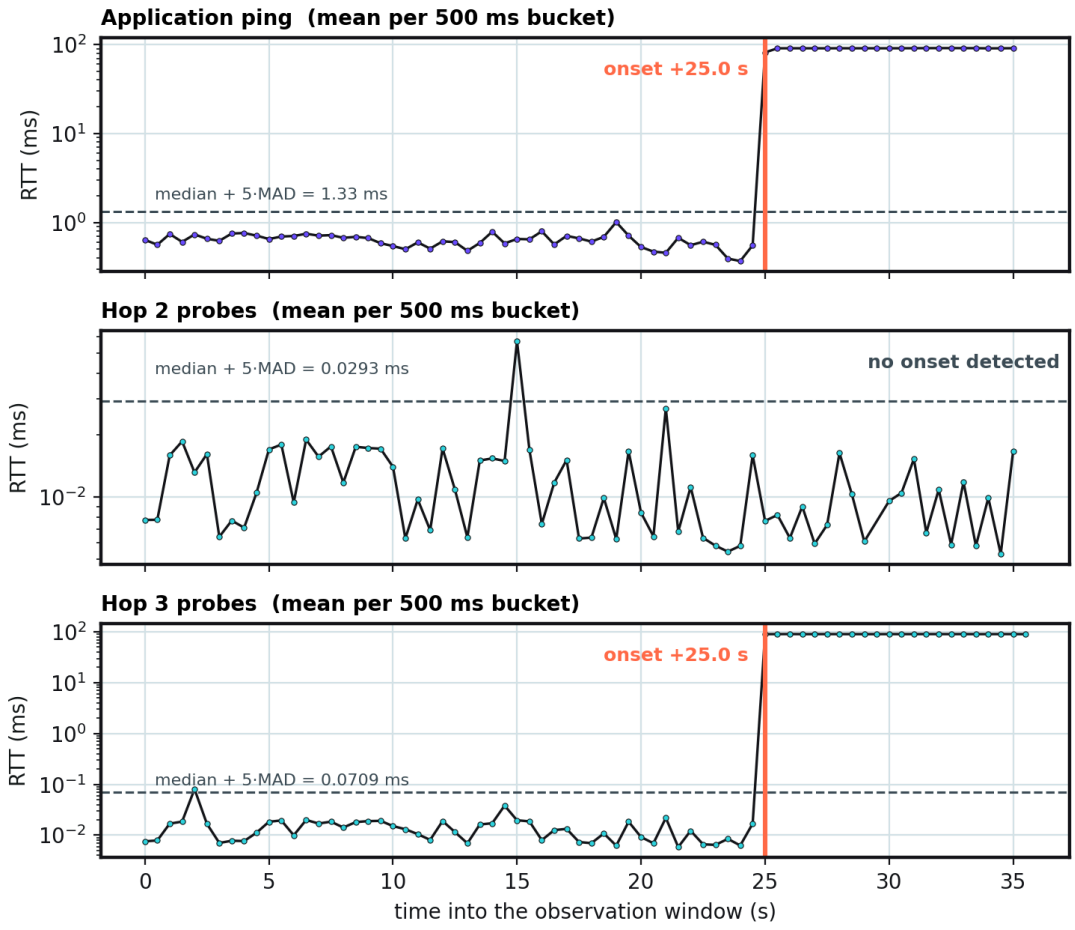


Figure 4. Measured series from `experiments/demo-run-01` (500 ms buckets), recomputed with the project's own modules; the onsets drawn equal those stored in `evidence_demo.json` (checked by `submission/build/make_figures.py`). The application and Hop 3 both show an onset at +25.0 s. Hop 2 has one noisy bucket above its threshold near 15 s, which the three-in-a-row rule correctly ignores; its probes show no onset.

9.3 Timing alignment and downstream persistence

Let t_{app} be the application onset and t_k the onset of hop k (if any). Two onsets are **concurrent** when $|t_k - t_{app}| \le \Delta$ with Δ = **1.5 s** (three probe rounds; a documented default that has **not** been validated). The engine scans hops from the shallowest and reports the first hop c such that

1. hop c has an onset concurrent with the application's, **and**
2. every hop from c through the deepest measured hop also has a concurrent onset.

The second condition is the *persistence* test: a hop that spikes alone, with nothing downstream, is evidence *against* localizing there. The result is the range `{min: c, max: deepest corroborating hop}`. If there is no application onset, nothing is localized. If there is an application onset but no hop satisfies both conditions, the result is "not localized".

9.4 Evidence representation, confidence and interpretation

A hop is marked **unknown** if, among the buckets in which it was probed, at least 50% received no response at all (`UNKNOWN_FRACTION_THRESHOLD = 0.5`, fixed, not tuned). The confidence claim is the following **fixed heuristic**:

$$\textit{score} = \textit{corroborating fraction} \times (1 - |\textit{unknown hops}| / 5) \quad \text{high} \ge 0.8 \cdot \text{medium} \ge 0.5 \cdot \text{low} > 0 \cdot \text{otherwise insufficient}$$

What the confidence score is, and is not

- It is **not a probability** and has not been calibrated. It does not measure how strong the signal was.
- Because the persistence rule already requires every downstream hop to corroborate, the corroborating fraction is **1.0 whenever a range is reported**, so in practice the level depends only on how many hops were unknown: 0 unknown → 1.0 (*high*); 1 → 0.8 (*high*); 2 → 0.6 (*medium*); 3 → 0.4 (*low*).
- The stored calibration tables show the weakness. In the independent Phase 4 run, the four *high*-confidence claims had an observed accuracy of only 0.5 (two of them were confident claims on clean controls); the Brier score was 0.432 (n = 5). In the exposed Phase 4b set the Brier score was 0.04 (n = 4), a figure from a small, inspected set and not evidence of calibration.

Interpretation then maps the record to one of the four locked sentences (Section 4.2): only *high* confidence yields "Degradation appears to begin at or after Hop N"; *medium* or *low* yields "Partially localized".

9.5 Limitations of the methodology

- **Global baseline.** The threshold uses the median and MAD of the *whole* series. It assumes the fault covers a *minority* of the window so that the median still represents "normal". If a fault covers most of the window, the median sits inside the faulted region and the detector reports no onset. The demo script documents a first version with a 10 s baseline and 25 s fault that produced exactly this false negative; the recorded demo therefore uses a 25 s baseline and runs in the detector's *favourable regime*.
- **No "insufficient data" state.** "No onset" cannot be distinguished from "the detector could not establish a baseline".
- **Near-silent series.** On sub-millisecond hop series the MAD can be tiny, so the threshold becomes an almost-zero absolute gap; this was the diagnosed cause of clean-control false positives in Phase 4 and the motivation for requiring a sustained run.
- **Resolution.** Onsets are quantised to 500 ms, and the 1.5 s alignment window is fixed and unvalidated.
- **RTT as the only observable.** No candidate localized any loss or bandwidth trial. The project records, as an open and unresolved problem in `frozen_config_4b.json`, that RTT-based onset detection may be the wrong observable for these two fault types; this has not been tested further.

- **Correlation, not causation.** Concurrent onsets are timing evidence only.

9.6 Algorithm selection (Phases 4 and 4b)

Phase 4 implemented five detectors behind one interface (`detect_onset(series, t)`) and compared them on a 12-trial tuning set; the selection rule was within-one-hop accuracy first, false attribution second, Brier score third. `mad` ranked first and was frozen with `k_mad = 5.0`.

Table 9.1 Phase 4 tuning-set scoreboard (12 tuning trials: 8 faults, 4 clean controls; from `evaluate/frozen_config.json`)

Detector	Within ±1 hop	Exact hop	False attribution	Clean-control FP	Recall	Brier (n)
threshold (strawman)	0/0	0/0	0/12	0/4	0/8	n/a (0)
mad (selected)	4/4	2/4	2/12	0/4	4/8	0.040 (4)
cusum	4/4	3/4	2/12	1/4	4/8	0.160 (5)
bocpd	3/4	3/4	1/12	0/4	4/8	0.250 (4)
pelt	2/3	1/3	2/12	0/4	3/8	0.333 (3)

After `mad` missed the false-attribution target on the held-out set (Section 12.1), Phase 4b inspected the artifacts, diagnosed the mechanism (Section 9.5), defined three variants, and compared them on a **new** 10-trial tuning set (4 clean controls, 2 loss, 2 bandwidth, 1 jitter, 1 latency).

Table 9.2 Phase 4b tuning-set scoreboard (10 new tuning trials; from `evaluate/frozen_config_4b.json`)

Detector	Within ±1 hop	Exact hop	False attribution	Clean-control FP	Recall	Brier (n)
mad_v1_frozen (Phase 4)	2/2	1/2	2/10	1/4	2/6	0.333 (3)
mad_absfloor	2/2	2/2	0/10	0/4	2/6	0.000 (2)
mad_sustained (selected)	2/2	2/2	0/10	0/4	2/6	0.000 (2)
mad_largest_run	2/2	2/2	1/10	1/4	2/6	0.333 (3)

`mad_absfloor` and `mad_sustained` tied exactly. The automated ranking picked `mad_absfloor` only through sort order; the tie was broken **manually** in favour of `mad_sustained` because requiring persistence encodes "sustained degradation" directly, whereas `mad_absfloor` relies on a fixed 0.5 ms constant. Both are disclosed as equally supported by the tuning data. On this tuning set, **loss and bandwidth were 0/2 each for every candidate**.

10. Dashboard and User Interaction

All screenshots in this section and the appendix were captured on 27 October 2026 from the **current production build** of `frontend/` (`npm run build`, served locally by `npm run preview` on 127.0.0.1) in a real browser. They show stored data only; no figure was drawn by hand.

10.1 Trial selection

The page opens on a persistent *Research prototype* banner that states the validation status, then a headline and a **trial picker**. Each button loads one stored presentation bundle from the project's own experiments: `demo-run-01` (a live demo run), `p4-held-01-clean1` (a clean control), `p4-held-04-latency-hop2-moderate` and `p4-held-12-unknown-plus-impair` (two held-out trials). The *Guided explanation* toggle is **off** by default.

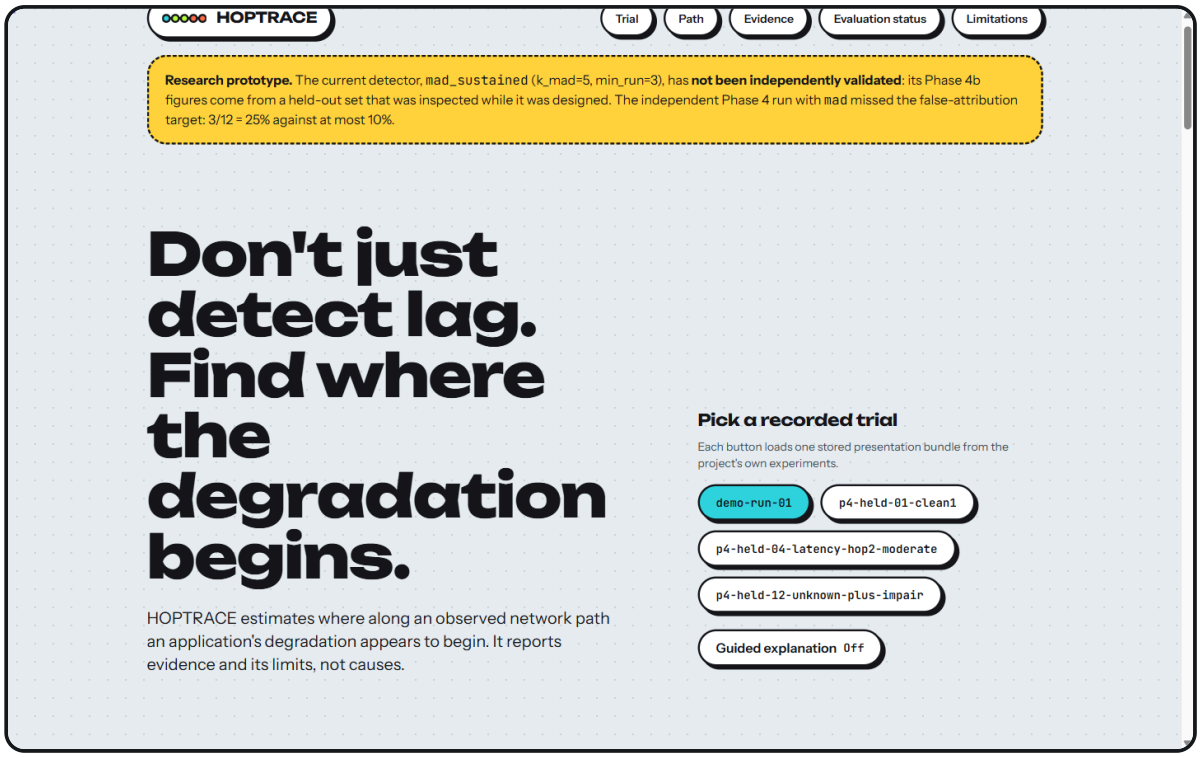


Figure 5. Landing view: validation banner, headline, trial picker and the guided-explanation toggle (Off).

10.2 RTT visualisation: the symptom first

The first section shows only what the **application** measured: its own ping RTT on a logarithmic axis, the stored mean before and after the detected onset (dashed violet lines), the onset marker, and the number of ping samples. It is labelled as the symptom, not the location.

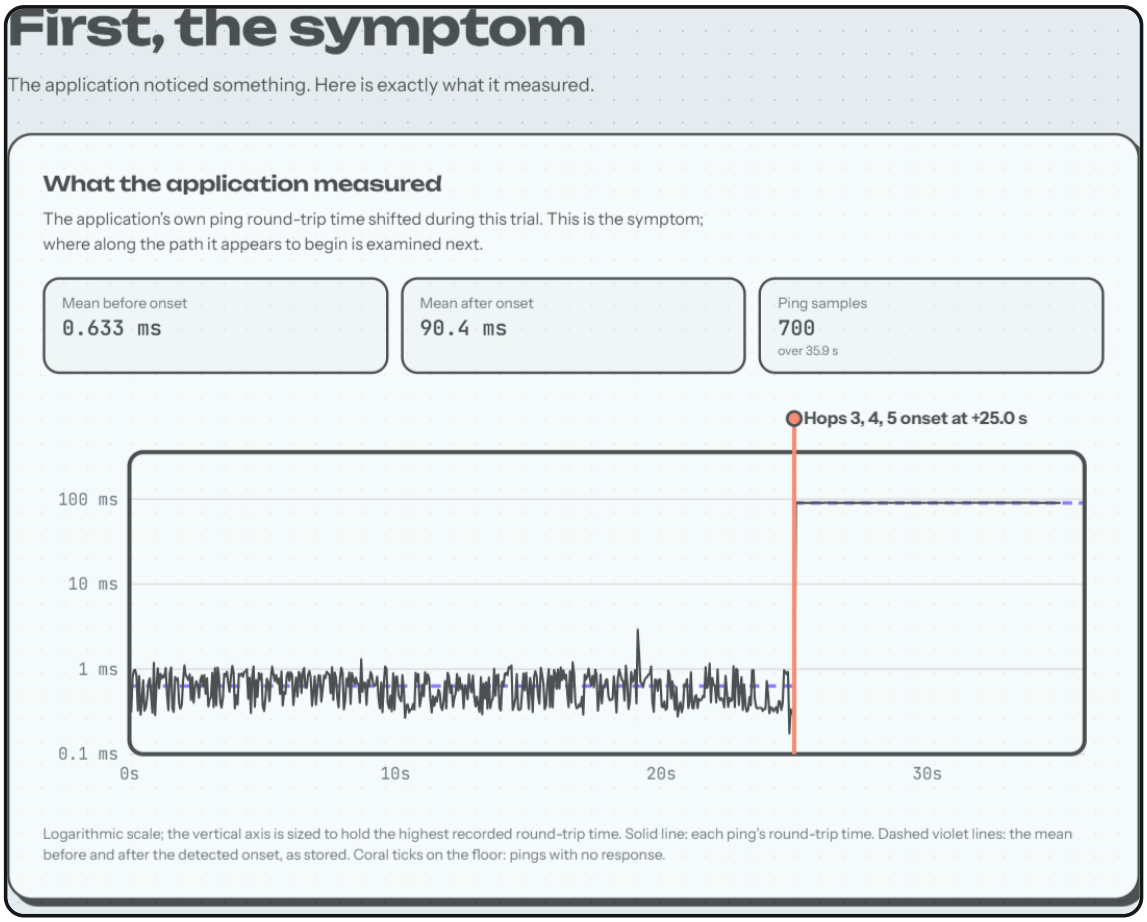


Figure 6. Application RTT for `demo-run-01`: mean 0.633 ms before the onset and 90.4 ms after, over 700 pings and 35.9 s; onset at +25.0 s.

10.3 Path and hop inspection

The next section draws the path (client, Hops 1-4 as routers, Hop 5 as the server). Each hop carries a status taken from the stored bundle: **corroborated onset** (coral), **no corroborated onset** (lime; *not proof of health*) or **unknown** (yellow, hatched). Selecting a hop opens the **hop inspector**: mean RTT before and after that hop's onset, probe samples, the onset time of the hop and of the application, the stored evidence sentences and measured facts, and the raw stored hop record. A hatched link means an onset persists across both neighbouring hops. The moving packets are **illustrative only**: the data contain no packet-level observations, and the page says so beside a *Pause packet animation* control.

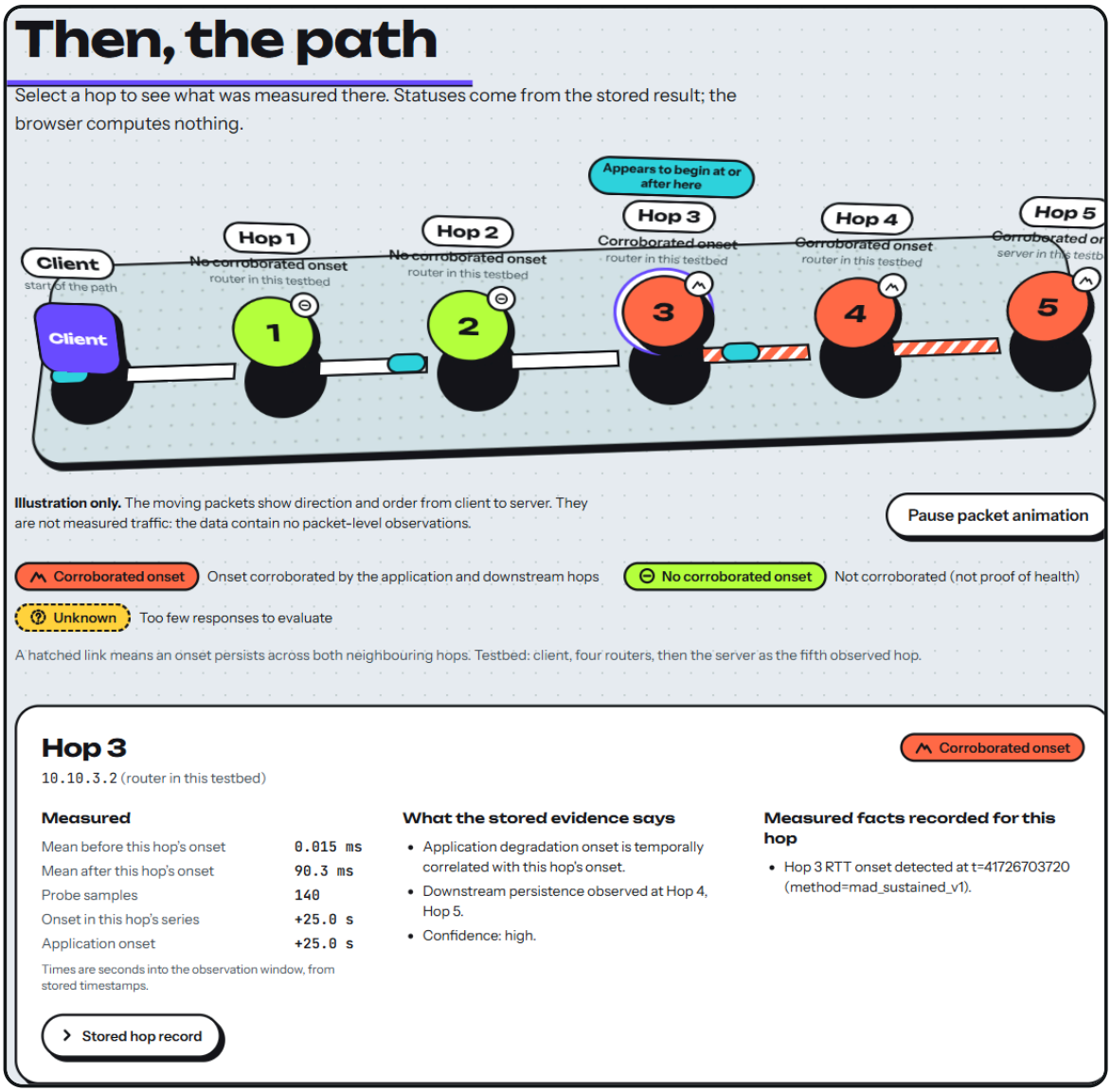


Figure 7. `demo-run-01` : Hops 1 and 2 show no corroborated onset; Hops 3, 4 and 5 are corroborated, with "Appears to begin at or after here" on Hop 3. The inspector shows Hop 3: 0.015 ms before and 90.3 ms after its onset at +25.0 s (140 probe samples). The moving packets are an illustration, not measured traffic.

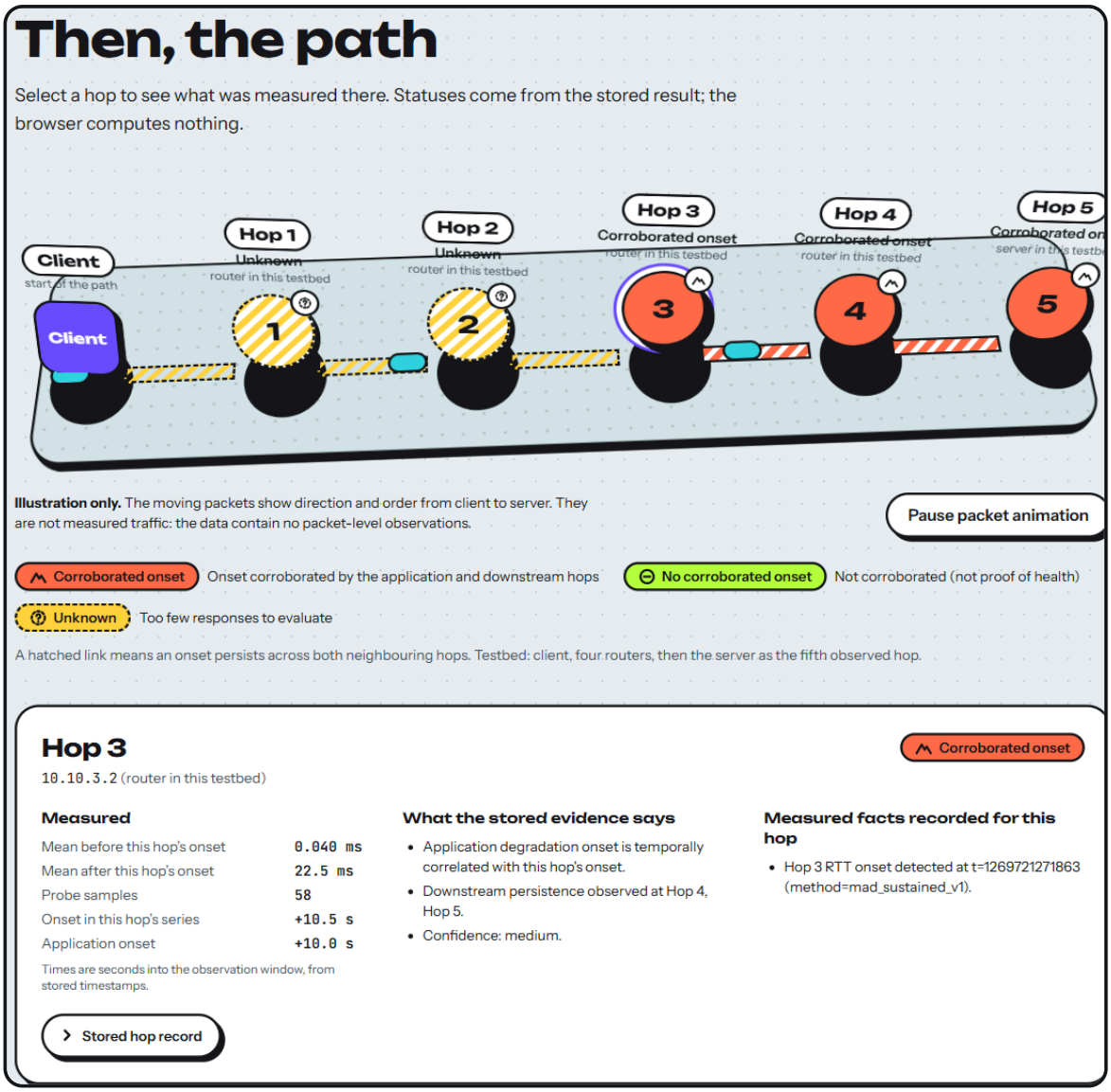


Figure 8. `p4-held-12-unknown-plus-impair`: Hops 1 and 2 returned too few responses and are shown as **unknown**, not healthy. The result is *partially localized* with *medium* confidence (Hop 3 inspected: 0.040 ms before, 22.5 ms after, onset +10.5 s against an application onset at +10.0 s, 58 probe samples). In the stored evaluation the first hop the injected fault could affect was Hop 2, which was itself unknown (no responses), so it could not be corroborated; the detector reported Hop 3 and the trial was scored as a false attribution (Section 12.2).

10.4 Stored evidence, kept apart by kind

Below the boundary statement ("Degradation appears to begin at or after Hop 3. This is the earliest hop whose onset is corroborated in the stored data. It is a statement about when changes were observed, not a finding about the reason for them.") the page separates the evidence into **measured facts**, **inference**, **temporal correlation** and **unknown**, and shows the **stored confidence** with its limits: level and score, an explicit statement that it is a fixed heuristic and not a probability, the corroborating and unknown hops, and the completeness fraction.

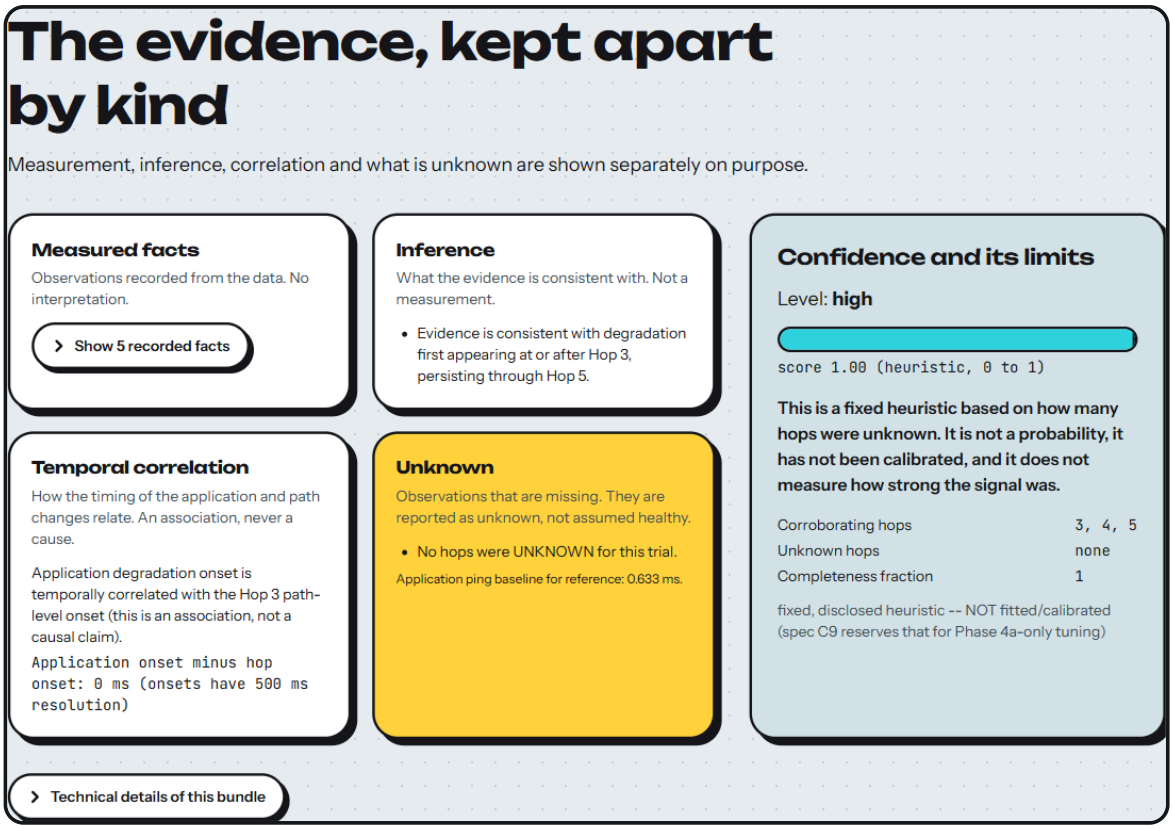


Figure 9. Evidence panels for `demo-run-01`. The unknown panel states that missing observations are reported as unknown, "not assumed healthy". The confidence panel states that the score is "a fixed heuristic … not a probability … not calibrated".

10.5 Guided explanation mode

The *Guided explanation* toggle (off by default) adds a plain-language walkthrough built **only from the selected bundle**: it states the symptom the application saw, what each hop shows, what the stored result infers, how far to trust it, and what the result does **not** show. A five-step walkthrough (*Previous / Next / Exit*) moves through the same material and can scroll to the matching part of the page. A clean-control trial is explained in distinct wording (no degradation detected, which does not prove health).

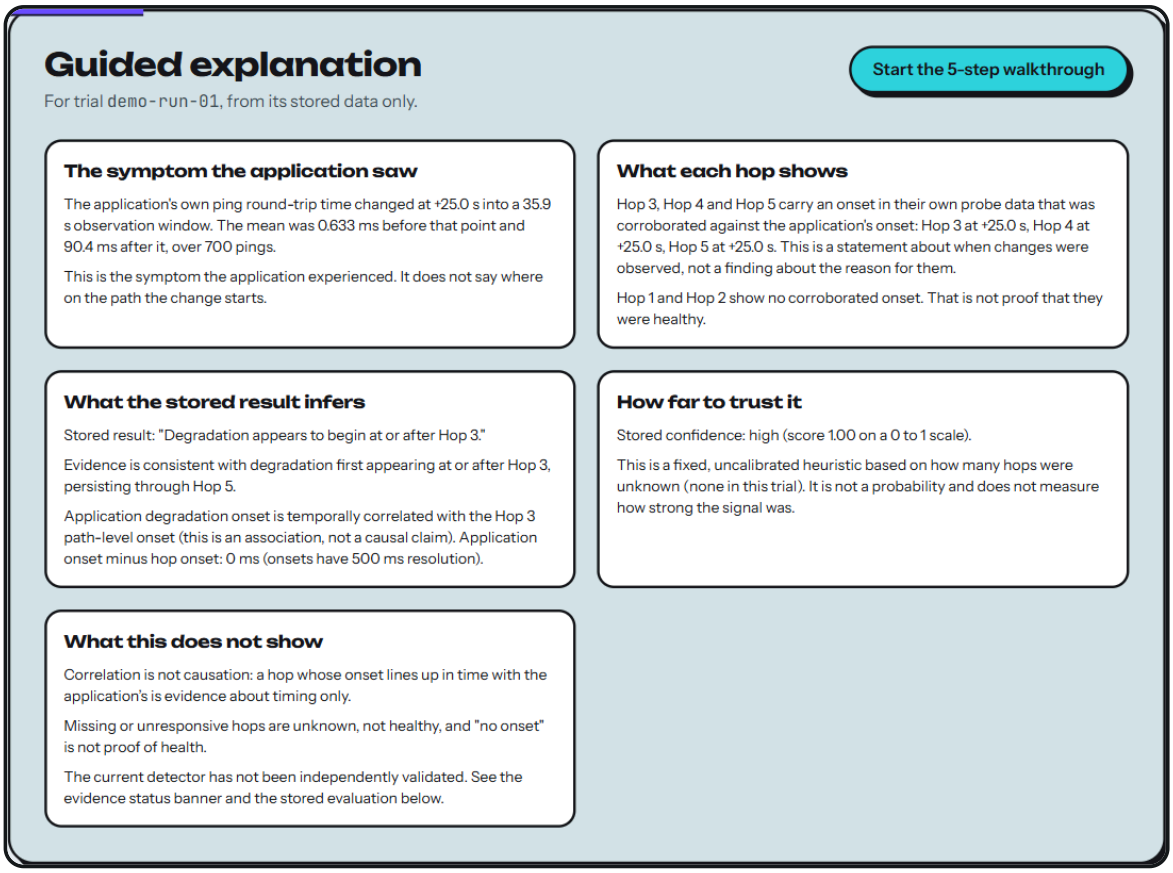


Figure 10. Guided explanation overview for `demo-run-01`, including "What this does not show": correlation is not causation; unknown is not healthy; the current detector has not been independently validated.

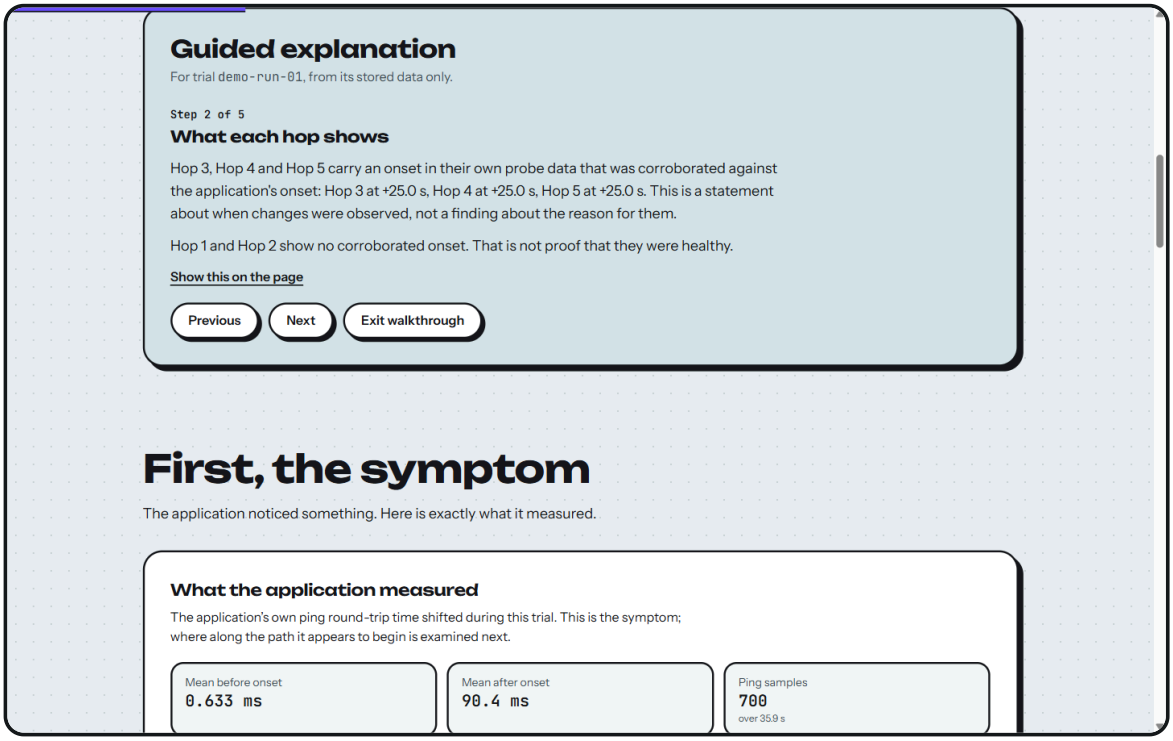


Figure 11. Walkthrough step 2 of 5, "What each hop shows", with *Previous*, *Next* and *Exit walkthrough* controls. The guided-explanation components are in the working tree of the repository and are covered by `guided.test.tsx` (11 tests) and by `check:preview`.

Further captures (stored evaluation, limitations, the no-degradation result and the partially-localized result) are in Appendix D.

11. Testing and Verification

This section reports checks that were **actually run on 27 October 2026** for this report, in the environments named. Earlier counts quoted in other project documents were not reused. Unit tests, integration and browser checks, and the experimental evaluation are kept separate; the experimental results are in Section 12.

Table 11.1 Verification layers and their results

Layer	Command	Result (27 Oct 2026)
Python unit tests	`python3 -B -m unittest discover` per test directory	**117 tests, all OK** (WSL2 Ubuntu 24.04, Python 3.12.3, unprivileged user)
Repository hygiene	`python3 tools/check_repo_hygiene.py`	OK: 59 source files, no per-user paths, no CR characters, `.gitattributes` present
Frontend typecheck	`npm run typecheck`	passes (`tsc -b --force` , strict)
Frontend lint	`npm run lint`	passes with `--max-warnings 0`
Frontend tests	`npm test`	**310 tests in 14 files, all passed** (vitest 5.0.4, jsdom)
Production build	`npm run build`	built; one ~511 kB script (not code-split; vite warns about chunk size)
Distribution safety	`npm run verify:dist`	OK: 21 files (4 bundles, 2 evaluation projections, 1 index); no raw traces, databases, labels, credentials or remote runtime dependencies
Real-browser layout	`npm run check:layout`	OK: 40 checks (10 widths × 4 real trials): no horizontal overflow, no overlapping hop labels or text, all hop controls ≥ 44 px
Real-browser interaction	`npm run check:preview`	OK: 4 trials switched in one session against `vite preview` ; every bundle HTTP 200 JSON and rendered; packet pause/resume; guided walkthrough steps 1-5; prohibited URLs return no data
Live testbed	`setup_topology.sh` + `verify_topology.sh`	PASS: 200/200 packets (0% loss); five hops in order, twice (Section 8.5)

11.1 Python unit tests: what they cover

Directory	Tests	Modules under test
`prober/tests`	9	`hopsample.py` (record shape and (de)serialization), `raw_probe.py` (checksum and ICMP parsing)
`app/tests`	10	`ws_protocol.py` (RFC 6455 framing and handshake), `apprttsample.py`
`correlate/tests`	38	`synchronize`, `onset`, the Phase 4 and Phase 4b candidate detectors, `align`, `evidence`
`evaluate`	16	`metrics.py` (false attribution, hop matching, Wilson interval, summaries)
`present/tests`	29	`interpret` (locked sentences, forbidden causal phrases), `technical_view`, `path_bundle` (hop status re-expresses stored evidence; run against real recorded trials)
`faultctl/tests`	15	`controller.py` (authorized targets, `tc` argument construction; `tc` is mocked) and `groundtruth.py` (store, with the database path patched to a temporary directory)
Total	**117**	

The tests were run from a Windows-mounted path inside WSL2 (`/mnt/c/…`) as a normal user (uid 1000) with bytecode writing disabled; the `faultctl` tests print a stray event UUID, which is expected. They do not need root and cannot write the real ground-truth database. They do **not** exercise the live network testbed.

11.2 Frontend tests: what they cover

Test file	Tests	Covers
`vite/dataAllowlist.test.ts`	49	Which stored files may be served; traversal and non-allowlisted paths refused
`vite/devServer.test.ts`	40	Starts the real dev server and probes about forty traversal / ground-truth / trace / config URLs
`vite/previewServer.test.ts`	38	Builds the site and fetches all four bundles over HTTP from `vite preview`
`vite/devSourceGuard.test.ts`	29	Dev server hides its own non-source files
`src/components/RttChart.test.tsx`	34	RTT chart geometry and rendering
`src/stageC.test.tsx`	26	The visual experience: path, hop inspector, statements, accessibility
`src/App.test.tsx`	19	Application states: loading, error, trial switching
`src/data/loader.test.ts`	19	Typed data loader and error handling
`src/design/tokens.test.ts`	16	WCAG AA contrast of the design tokens (reads the CSS)
`src/data/schema.test.ts`	14	Bundle schema validation (Zod) against real artifacts
`src/guided.test.tsx`	11	Guided explanation and walkthrough
`src/components/primitives/primitives.test.tsx`	9	Badge, pill, sticker and disclosure components
`src/data/evaluationStatus.test.ts` , `src/styles/packets.test.ts`	3 + 3	Evaluation status projection; packet animation rules
Total	**310**	14 files

11.3 Reproducible commands

```
# Python tests (WSL2, from the repository root, no root needed)
for d in prober/tests app/tests correlate/tests evaluate present/tests faultctl/tests; do
  PYTHONDONTWRITEBYTECODE=1 python3 -B -m unittest discover -s "$d" -p 'test_*.py' || break
done
python3 tools/check_repo_hygiene.py

# Frontend (from frontend/, Node ⩾ 22.12)
npm ci
npm run verify          # typecheck, lint, test, build, verify:dist
npm run check:layout    # needs Chrome or Edge
npm run check:preview   # needs Chrome or Edge
```

Not verified in this report

- **The full live demo** (`demo/run_demo.sh`) was not re-run: it creates identities, rebuilds the topology and **overwrites** the recorded `experiments/demo-run-01` . The recorded trial is documented as produced by an earlier real run.
- **The historical evaluation scripts** (`evaluate/tuning_analysis*.py` , `held_out_evaluation*.py` , `diagnose_phase4b.py` , `pilot_analysis.py`) were deliberately **not** re-run: they read ground truth and overwrite the frozen configurations and result files.
- **Offline operation** of the dashboard (network disabled) and **cross-browser or real-device** behaviour were not tested.
- A **clean clone** of a committed revision has not been tested; the tests above ran in the development working tree, which at the time also held uncommitted guided-explanation changes.

12. Experimental Results and Analysis

All figures are quoted verbatim from `evaluate/held_out_results.json` (Phase 4) and `evaluate/held_out_results_4b.json` (Phase 4b); the report build asserts them. The evaluation sets contain 12 trials each: 8 fault trials and 4 clean controls. **Every rate below rests on a very small sample and should be read as rough.**

Headline findings, stated plainly

- **Phase 4 failed its false-attribution target: 3/12 = 25%, against a target of at most 10%.** This is the only independent held-out result.
- Phase 4b's **1/12 = 8.3%** was measured on the **same held-out set after its traces had been inspected** during design. It is a historical record and **not** independent validation.
- Loss and bandwidth faults were **not localized**: 0 of 4 on the exposed held-out set and 0 of 4 on the Phase 4b tuning set.

12.1 Phase 4: detector `mad` on the held-out set (independent)

The Phase 4 configuration (`mad` , `k_mad = 5.0` , Δ = 1.5 s) was frozen from the tuning set and scored once on the 12 held-out trials **before any retuning**.

Table 12.1 Phase 4 summary (`held_out_results.json`)

Metric	Result	Wilson 95% interval	Reading
False attribution (target ≤ 10%)	3/12 = 25.0%	8.9% – 53.2%	**Target missed. Two confident claims on clean controls, one wrong confident claim.**
Clean-control false positives	2/4 = 50%	15.0% – 85.0%	Two of four clean controls received a high-confidence claim.
Exact first hop (localized trials)	2/3 = 66.7%	20.8% – 93.9%	Only three fault trials produced a claim.
Within ±1 hop (localized trials)	3/3 = 100%	43.8% – 100%	Denominator is only the trials that were localized.
Recall of injected faults	3/8 = 37.5%	n/a	Five of eight faults produced no claim.
Precision of claims	3/5 = 60%	n/a	
Brier score of confidence	0.432 (n = 5)	n/a	Poor; the four *high*-confidence claims were 50% accurate.

12.2 Phase 4b: detector `mad_sustained` on the same set (exposed)

Why this is not independent validation

`evaluate/diagnose_phase4b.py` loaded nine of the held-out traces together with their ground-truth labels and was written **before** the Phase 4b detector variants existed; the revised configuration was then scored on that very set. The design therefore saw the data it was later scored on. The evidence ledger marks `p4-held-*` permanently **EXPOSED**: it may be used for historical analysis and transparent reporting, but never to claim independent validation of a revised algorithm. The `gate_check.overall_pass = true` field in the result file is a computed comparison against thresholds, not a validation.

Table 12.2 Phase 4b summary (`held_out_results_4b.json` , exposed set)

Metric	Result	Wilson 95% interval	Reading
False attribution (target ≤ 10%)	1/12 = 8.3%	1.5% – 35.4%	Numeric threshold met on **exposed** data. The one false attribution is `p4-held-12` .
Clean-control false positives	0/4 = 0%	0% – 49.0%	The sustained-run rule removed the Phase 4 clean-control false positives.
Exact first hop (localized trials)	3/4 = 75%	30.1% – 95.4%	
Within ±1 hop (localized trials)	4/4 = 100%	51.0% – 100%	Four localized trials.
Recall of injected faults	4/8 = 50%	n/a	Four of eight faults still produced no claim (all four loss and bandwidth trials).
Brier score of confidence	0.040 (n = 4)	n/a	From four claims on an inspected set; not evidence of calibration.

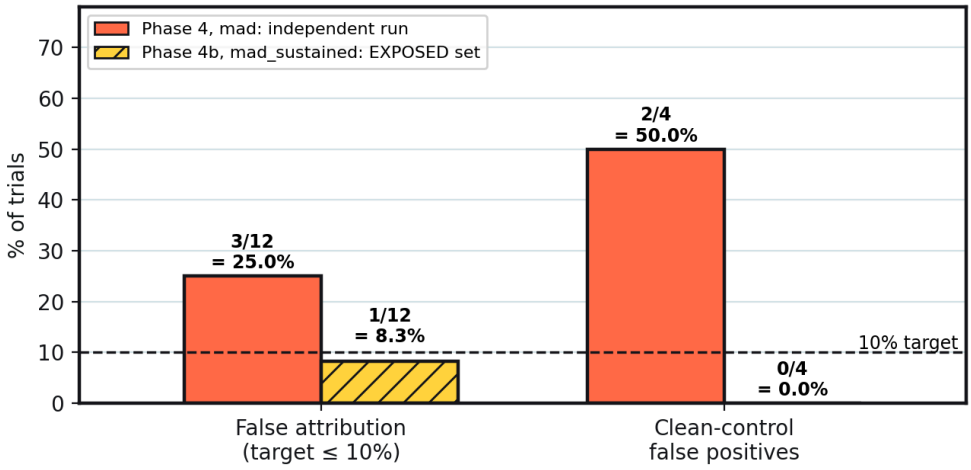


Figure 12. False attribution and clean-control false positives. The hatched Phase 4b bars come from an **exposed** set and must not be read as independent confirmation.

12.3 Per-trial outcomes on the held-out set

"Expected hop" is the first hop the injected fault could affect (`min(hop_target + 1, 5)`). A **FA** mark means the claim was scored a false attribution. These labels already appear in the repository's result files.

Table 12.3 Held-out trials: Phase 4 (`mad`) against Phase 4b (`mad_sustained`); reported hop range with confidence level

Trial	Class	Expected hop	Phase 4 (independent)	Phase 4b (exposed)
`01-clean1`	clean control	none	**Hops 1-5, high FA**	no claim
`02-clean2`	clean control	none	no claim	no claim
`03-clean3`	clean control	none	**Hops 1-5, high FA**	no claim
`04-latency-hop2-moderate`	latency	3	3-5, high ✓	3-5, high ✓
`05-jitter-hop3-mild`	jitter	4	4-5, high ✓	4-5, high ✓
`06-loss-hop4-severe`	loss	5	**no claim (miss)**	**no claim (miss)**
`07-bandwidth-hop1-moderate`	bandwidth	2	**no claim (miss)**	**no claim (miss)**
`08-jitter-hop2-transient`	jitter	3	**no claim (miss)**	3-5, high ✓
`09-bandwidth-hop3-transient`	bandwidth	4	**no claim (miss)**	**no claim (miss)**
`10-loss-hop1-mild`	loss	2	**no claim (miss)**	**no claim (miss)**
`11-unknown-hop1`	clean control, Hop 1 silent	none	no claim	no claim
`12-unknown-plus-impair`	jitter, Hops 1-2 silent	2	**3-5, medium FA**	**3-5, medium FA**

12.4 Analysis

- **What changed between Phase 4 and 4b** (on the exposed set): the two clean-control false positives disappeared and one jitter trial (`08`) became localized. Nothing changed for loss or bandwidth, and `p4-held-12` remained a false attribution. The improvement is consistent with the diagnosed mechanism (single noisy buckets crossing a near-zero absolute threshold), but it cannot be separated from the fact that the design saw these traces.
- **Latency and jitter** were localized when a claim was made (latency 1 of 1; jitter 3 of 3 under Phase 4b, one of them a false attribution). **Loss and bandwidth** were localized in 0 of 4 held-out trials (by both detectors) and 0 of 4 Phase 4b tuning trials (by all four candidates). A possible reason, that bucketed RTT may be the wrong observable for these faults, is recorded in the project as an open problem, not as a finding.
- **The false attribution in `p4-held-12`** occurred where the first affected hop (Hop 2) was itself unknown; the detector reported the next hop. The system correctly lowered confidence to *medium* and shows Hops 1 and 2 as unknown, but under the project's own definition (a confident claim that is wrong) it is a false attribution.
- **Small samples.** With 12 trials, one trial moves a rate by 8.3 percentage points; "within ±1 hop 4/4" has a 95% interval of 51% to 100%; four clean controls cannot distinguish 0% from nearly 50% false positives. The specification's 120-trial tuning set and 288-cell evaluation matrix were not run.

12.5 The recorded demo run (not an evaluation)

`demo-run-01` is a recorded real run: a 90 ms latency impairment on the link leaving Hop 2, injected 25 s into a 35 s capture. The frozen `mad_sustained` pipeline reported *"Degradation appears to begin at or after Hop 3"*, high confidence (score 1.0), corroborating Hops 3, 4 and 5, with the application and Hop 3 onsets both at +25.0 s (Figure 4, Figure 7). From the recorded files: Hop 3 probes averaged **78.53 ms** over the last third of the faulted

capture and **0.012 ms** in a fresh 10-round probe after the fault was cleared (`recovery_hop.jsonl`). The scenario was chosen in the detector's favourable regime (fault covering a minority of the window) and illustrates the pipeline; it is **not** evidence about detector accuracy.

13. Limitations and Threats to Validity

Each limitation below is supported by a file in the repository; the source is given in brackets.

#	Limitation	Detail and consequence
1	**No independent validation of the current detector**	`mad_sustained` was never scored on data that its design had not seen. The one independent run (Phase 4, `mad`) missed its target at 3/12 = 25%. A sealed evaluation (spec Phase 5) has not been run. [`docs/EVIDENCE_LEDGER.md`]
2	**Exposed held-out set**	`p4-held-*` is permanently exposed; Phase 4b results on it are a historical record. The set was never sealed or made read-only. [ledger]
3	**Loss and bandwidth are not localized**	0/4 on the exposed held-out set (Phase 4 and 4b detectors) and 0/4 on the Phase 4b tuning set (all four candidates). Whether bandwidth limits are observable at all with the current light probe traffic is unproven; bandwidth is not de-scoped, and no investigation is authorized yet. [`frozen_config_4b.json`]
4	**Confidence is an uncalibrated heuristic**	A fixed formula based almost entirely on how many hops were unknown; the corroborating fraction is 1.0 whenever a range is reported. It is not a probability and does not reflect signal strength. Phase 4's *high* claims were 50% accurate. [`correlate/evidence.py`]
5	**Unknown hops cannot be assumed healthy**	A hop with too few responses is unknown, and "no corroborated onset" is not proof of health. A silent hop can also prevent localization, as in `p4-held-12`.
6	**Timing correlation is not causation**	HOPTRACE says when changes appear along an observed path. It does not establish which device, link or configuration caused them.
7	**"No onset" is ambiguous**	It can mean healthy, a fault covering most of the window (global median/MAD baseline), or too little data. There is no "insufficient data" state. [demo script; ledger]
8	**Fixed, unvalidated parameters**	The 1.5 s alignment window, the 500 ms bucket width (which also limits onset resolution), `k_mad = 5`, `min_run = 3` and the 50% unknown threshold are fixed choices. [`align.py`, `pipeline.py`]
9	**Very small samples**	12 + 12 + 10 trials against a specified 120-trial tuning set and 288-cell matrix. Confidence intervals are wide (Section 12). [`evaluate/power_calc.py`]
10	**Controlled single-path testbed**	One linear path, no ECMP, one application protocol, and ICMP error rate limiting deliberately switched off in the router and server namespaces. Behaviour on real routers and networks has not been evaluated. [`setup_topology.sh`]
11	**Ground-truth isolation is a permission test**	Root can read the database; historical trial directory names encode the injected fault; evaluation result files contain labels and are world-readable on a Windows-mounted checkout. [README]
12	**Phase order and labels**	The UI and demo (Phase 6) were built before the Phase 5 evaluation, which the specification would not have allowed; several code comments use pre-numbering labels. [ledger]
13	**Demo scenario is favourable**	The demo fault covers a minority of the window by construction; the first version (a majority) produced a false negative. [`demo/demo_script.md`]
14	**Dashboard limits**	It shows four stored trials, not live measurements; it has not been tested offline or across browsers; hop roles are a frontend constant, not a data field; there is no real "not localized" bundle and none was invented; the page is one ~500 kB script. [`frontend/README.md`]
15	**Documentation drift**	The evidence ledger counts six exploratory `p3-*` trials, but five directories exist in `experiments/`; the figure used in this report is the directory count.

14. Installation and Execution Guide

14.1 Environments

Part	Verified environment (27 Oct 2026)	Not verified
Testbed, probing, telemetry, Python tests	Windows 11 + **WSL2 Ubuntu 24.04** (kernel 6.18.33.2-microsoft-standard-WSL2), Python 3.12.3, iproute2 6.1.0	Native Linux, macOS, WSL1, other Python versions, a fresh Ubuntu install
Dashboard and its checks	**Windows 11**, Node 24.14.1, npm 11.11.0, Chrome (for the browser checks)	Node below 22.12; other OSes and browsers

Windows-native Python is **not** supported (the code uses `time.clock_gettime_ns` and Linux network namespaces). Raw sockets, namespaces and `tc` need root. Network namespaces and `tc` state are **lost whenever WSL restarts**; rebuild the topology each session.

14.2 Get the code

```
git clone https://github.com/KrishUsesGit/F2_Team24.git F2_Team24
cd F2_Team24
```

Clone to an ordinary path. A OneDrive-synced folder risks sync conflicts and locks on `.git` and `node_modules`. Inside WSL a clone on the Linux filesystem (for example `~/hoptrace`) is preferable to `/mnt/c/…`; the ground-truth database is created under `/var/lib/hoptrace` in any case, never in the repository.

14.3 Run the unit tests (safe, no root)

```
# in WSL2, from the repository root, as a normal user
for d in prober/tests app/tests correlate/tests evaluate present/tests faultctl/tests; do
  PYTHONDONTWRITEBYTECODE=1 python3 -B -m unittest discover -s "$d" -p 'test_*.py' || break
done
python3 tools/check_repo_hygiene.py
```

Expected: six `OK` blocks reading `Ran 9 tests`, `Ran 10`, `Ran 38`, `Ran 16`, `Ran 29`, `Ran 15` (117 in total), then `OK: no per-user paths, no CR characters, .gitattributes present`.

14.4 Build and verify the testbed (modifies the WSL system)

```
# as root in WSL2, from the repository root
sudo bash faultctl/setup_identities.sh        # creates hoptrace-truth / -engine / -eval, /var/lib/hoptrace
(0750),
                                              # and /etc/sudoers.d/hoptrace-truth (tc inside ns-r1..ns-r4
only)
sudo bash testbed/phase0/teardown_topology.sh # safe if nothing exists
sudo bash testbed/phase0/setup_topology.sh    # NOT idempotent: always tear down first
sudo bash testbed/phase0/verify_topology.sh
```

Expected (observed on 27 Oct 2026 for the last three commands): `topology up.`; `icmp_ratelimit calibration: 0 (ns-r1, expect 0)`; `200 packets transmitted, 200 received, 0% packet loss` with `PASS: zero-loss baseline`; then the hop sequence `10.10.1.2 10.10.2.2 10.10.3.2 10.10.4.2 10.10.5.2` twice with `PASS`. `setup_identities.sh` was not re-run for this report (the identities already existed on the verification machine).

14.5 Run the live demo

```
sudo bash demo/run_demo.sh      # as root in WSL2, about 60-90 s
```

The script rebuilds the topology, proves live that `hoptrace-engine` cannot read the ground-truth database, starts the app server, prober and client, injects a real 90 ms latency fault on Hop 2's egress after 25 s, runs the frozen detector as `hoptrace-engine`, writes `path_bundle.json`, clears the fault and measures recovery. **It overwrites** `experiments/demo-run-01/` (the recorded trial) and has no cleanup trap if it fails midway. **It was not re-run for this report;** the expected result is described in `demo/demo_script.md` and matches the recorded trial (*"Degradation appears to begin at or after Hop 3"*, high confidence, recovery from about 78.5 ms to about 0.01 ms at Hop 3). To keep the recorded trial, copy `experiments/demo-run-01/` elsewhere before running.

14.6 View the dashboard

```
cd frontend
npm ci                  # installs exactly what package-lock.json records
npm run verify          # typecheck, lint, test, build, verify:dist
npm run preview         # serves dist/ at http://127.0.0.1:4173/   (run `npm run build` first if you skip
verify)
```

Open `http://127.0.0.1:4173/`. For development with the same data use `npm run dev`. **Expected** (observed): `verify:dist OK: 21 files (4 bundles, 2 evaluation projections, 1 index); no raw traces, databases, labels, credentials or remote runtime dependencies.` The page must be served by `vite`; opening `index.html` directly does not load the data. The server must keep running: if it stops, every trial fails with "The data files could not be reached". The dashboard reads `../experiments` and `../evaluate` at build time, so keep `frontend/` inside the repository checkout. The legacy static page is served with `python3 -m http.server 8420` from the repository root (`/ui/index.html`).

14.7 Selecting and running experiments

- **The recorded trials** in `experiments/` can be analysed again without a testbed. The four trials shown by the dashboard are those with a `path_bundle.json`.
- **Run your own trial** on a built testbed (each command in the namespace shown; paths from the repository root, as root unless noted):

```
mkdir -p experiments/my-trial
ip netns exec ns-server python3 app/server.py --host 0.0.0.0 --port 8765 &
ip netns exec ns-client python3 prober/prober.py --dst 10.10.5.2 --src 10.10.1.1 --transport tcp_syn \
    --rounds 70 --interval 0.5 --out experiments/my-trial/hop.jsonl &
ip netns exec ns-client python3 app/client.py --host 10.10.5.2 --port 8765 --duration 35 \
    --out experiments/my-trial/app.jsonl &
# after ~25 s, inject a fault as the controller identity (hop 1-4; latency/jitter/loss/bandwidth):
su -s /bin/bash hoptrace-truth -c "cd faultctl && python3 controller.py inject --hop 2 --type latency --
delay-ms 90 --trial-id my-trial"
# when the trial ends, clear it:
su -s /bin/bash hoptrace-truth -c "cd faultctl && python3 controller.py clear-latest --hop 2 --trial-id my-
trial"
```

To analyse with the frozen configuration, mirror steps 5-6 of `demo/run_demo.sh` (`demo/_run_engine_pipeline.py`, then `present/path_bundle.py`). These manual commands are derived from the scripts' own argument parsers and docstrings; they were **not** executed for this report. Do not run the `evaluate/` scripts on new data without reading Section 11 first.

14.8 Troubleshooting

Symptom	Cause and fix
Only ~60% of probes get replies; hops look "unknown"	Kernel ICMP rate limiting. `setup_topology.sh` sets `icmp_ratelimit=0`; confirm with the "expect 0" line, and rebuild after a WSL restart.
`ip netns` shows nothing; `Cannot find device`; controller says "is the topology up?"	WSL restarted, so namespaces are gone. Run teardown then setup again.
Errors from `ip netns add` or `ip link add` about something already existing	`setup_topology.sh` is not idempotent. Run `teardown_topology.sh` first.
Permission errors from the prober or `ip netns exec`	Raw sockets and namespaces need root.
Ground-truth permission errors (expected for the engine); database missing	Run `setup_identities.sh`. Keep the database on the Linux filesystem: `/mnt/c` ignores `chmod` and cannot enforce isolation.
Dashboard: "The data files could not be reached"	The preview server stopped. Restart `npm run preview` and reload.
`npm` install is slow or conflicts	The repository is inside a OneDrive-synced folder. Clone elsewhere and run `npm ci` there. Node must be ≥ 22.12.
Port 4173 or 8420 already in use	Stop the other server, or pass a different port to `vite preview` / `http.server`.
Shell scripts fail with `\r` errors	The `.sh` files must keep LF endings; `.gitattributes` enforces this on checkout.
Detector reports "no significant degradation" for an obvious fault	The fault probably covers most of the window, so the global median is inside the fault (Section 9.5). Use a longer baseline.

14.9 Cleanup

```
sudo bash testbed/phase0/teardown_topology.sh                    # removes all six namespaces (verified)
# if a fault was left behind (only needed when namespaces still exist):
sudo ip netns exec ns-r2 tc qdisc del dev veth-s3-a root         # example for Hop 2; use the interface
from Section 5.7
pkill -f app/server.py; pkill -f prober/prober.py; pkill -f app/client.py
```

Removing the three identities, the sudoers rule and `/var/lib/hoptrace` is optional and **permanently deletes recorded ground-truth events**; it was not exercised for this report (the scored labels also live in `evaluate/*.json`). The topology teardown above was verified.

15. Conclusion and Future Work

15.1 Conclusion

HOPTRACE demonstrates, on a controlled testbed, how an application's RTT and a path's per-hop RTT can be measured on a shared clock and compared to estimate **where along an observed path application degradation appears to begin**. The engineering is complete enough to run end to end: a six-namespace testbed, a raw-socket prober, a hand-written WebSocket application, a permission-isolated fault controller, a median/MAD correlation engine with typed evidence, an evidence-bounded interpretation layer and a dashboard that shows

stored results. The checks run for this report all pass: 117 Python tests, 310 frontend tests, strict typecheck, zero-warning lint, a verified production build and distribution, 40 real-browser layout checks, a four-trial browser session and a live topology check.

The **science** is less settled than the engineering, and the project states so. The only independent evaluation **missed** its false-attribution target (3/12 = 25% against at most 10%). The revised detector met the numeric target only on a set it had been designed against (1/12 = 8.3%), which is not validation. Loss and bandwidth faults are not localized. Confidence is a fixed heuristic. These are results, not omissions, and the interface repeats them wherever a result is shown.

15.2 Future work (recorded, not yet authorized)

1. Propose a feasible evaluation scope against the locked specification, with expected runtime and statistical limits, and obtain approval before any large run.
2. Implement spec Phase 5: a **sealed** evaluation set, an evaluation harness and a calibration check of the confidence heuristic.
3. Investigate whether controlled background traffic and a throughput-sensitive measurement make bandwidth impairment observable (and whether RTT is the wrong observable for loss).
4. Add an explicit **"insufficient data"** result state so that "no onset" is no longer ambiguous.
5. Validate or replace the fixed 1.5 s alignment window and the global-baseline assumption.
6. Test the dashboard with the network disabled, across browsers, and from a clean clone; add a versioned schema for the stored bundle.

16. References and Technical Documentation

Only sources that the implementation, the protocol code or the evaluation statistics actually rely on are listed.

Protocols and operating-system documentation

1. I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, December 2011. (Implemented by hand in `app/ws_protocol.py`.)
2. J. Postel, *Internet Control Message Protocol*, RFC 792, IETF, September 1981. (ICMP Time-Exceeded and Destination-Unreachable parsed in `prober/raw_probe.py`.)
3. R. Braden, D. Borman and C. Partridge, *Computing the Internet Checksum*, RFC 1071, IETF, September 1988. (Checksum for the hand-built IP and TCP headers.)
4. W. Eddy (ed.), *Transmission Control Protocol (TCP)*, RFC 9293, IETF, August 2022. (TCP-SYN probe transport.)
5. S. Hemminger, "Network Emulation with NetEm", *linux.conf.au*, 2005; and the Linux manual pages `tc-netem(8)`, `tc-tbf(8)`, `ip-netns(8)`, `veth(4)`, `network_namespaces(7)`, `clock_gettime(2)` (`CLOCK_MONOTONIC`) and the kernel `ip-sysctl` documentation for `icmp_ratelimit`.
6. Python 3 standard-library documentation: `socket`, `asyncio`, `statistics`, `sqlite3`, `unittest`.

Statistical and changepoint methods (candidates compared in Phase 4)

7. F. R. Hampel, "The influence curve and its role in robust estimation", *Journal of the American Statistical Association* 69(346), 1974. (Median absolute deviation as a robust scale.)
8. E. S. Page, "Continuous inspection schemes", *Biometrika* 41(1-2), 1954. (CUSUM.)
9. R. P. Adams and D. J. C. MacKay, "Bayesian Online Changepoint Detection", arXiv:0710.3742, 2007. (BOCPD candidate.)
10. R. Killick, P. Fearnhead and I. A. Eckley, "Optimal detection of changepoints with a linear computational cost", *JASA* 107(500), 2012. (PELT candidate.)

11. E. B. Wilson, "Probable inference, the law of succession, and statistical inference", *JASA* 22(158), 1927. (Wilson score interval, `evaluate/metrics.py`.)
12. G. W. Brier, "Verification of forecasts expressed in terms of probability", *Monthly Weather Review* 78(1), 1950. (Brier score.)

Project documents (in the repository)

13. `hoptrace-phase-0-5-spec.html`: the locked specification (phase numbering, constraints, `EvidenceRecord` definition).
14. `docs/EVIDENCE_LEDGER.md`: what each data set and result may be used for.
15. `demo/demo_script.md`, `frontend/README.md`, `README.md`, `AUDIT_REPORT.md` (2026-10-14 project audit).

Software used by the dashboard (documentation)

16. React (react.dev), Vite (vite.dev), Tailwind CSS (tailwindcss.com), Zod (zod.dev), Vitest (vitest.dev).

17. Appendix

A. Repository structure

```
.
├── README.md                      overview, scientific status, how to run
├── docs/EVIDENCE_LEDGER.md        what each data set / result may be used for
├── hoptrace-phase-0-5-spec.html   locked specification (also -architecture-review, -phase-0-plan)
├── testbed/phase0/                setup / verify / teardown scripts, lib/common.sh, impair/clear helpers
├── prober/   app/   faultctl/     measurement and fault-injection code, each with tests/
├── correlate/                     synchronize, onset*, align, evidence, pipeline (+ tests/)
├── present/                       interpret, technical_view, path_bundle, report (+ tests/)
├── evaluate/                      metrics, selection/scoring scripts, frozen configs, result JSON
├── demo/                          run_demo.sh, demo_script.md
├── experiments/                   40 trial directories (hop.jsonl, app.jsonl, trial_meta.json, ...)
├── frontend/                      React + TypeScript dashboard (package.json, package-lock.json, src/,
vite/, scripts/)
├── ui/                            legacy static page (kept for history)
├── tools/check_repo_hygiene.py    read-only repository check
└── submission/                    this report, the presentation, screenshots, figures, build scripts
```

B. Important configuration files

File	Purpose
`evaluate/frozen_config.json`	Phase 4: `mad`, `k_mad = 5.0`, `delta_align_ns = 1.5e9`, tuning set and scoreboard
`evaluate/frozen_config_4b.json`	Phase 4b: `mad_sustained`, `k_mad = 5.0`, `min_run = 3`, git commit `74c2395`, environment, the unresolved loss/bandwidth limitation
`evaluate/held_out_results.json`, `…_4b.json`	Scored held-out results (historical; **overwritten if the scoring scripts are re-run**)
`testbed/phase0/lib/common.sh`	Single source of namespace names and addressing
`faultctl/setup_identities.sh`	OS identities and the restricted sudoers rule
`frontend/package.json`, `package-lock.json`	Dependency manifest and exact lockfile; `engines.node ⩾ 22.12`
`frontend/vite.config.ts`, `vite/*.ts`	Data allowlist, dev-server binding and exposure guard
`.gitignore`, `.gitattributes`	No secrets, `*.db`, `node_modules`, `dist`; LF line endings and byte-exact evidence

C. Test commands

See Section 11.3. Summary: Python tests per directory with `unittest discover`; `python3 tools/check_repo_hygiene.py`; in `frontend/`: `npm run verify`, `npm run check:layout`, `npm run check:preview`.

D. Additional dashboard screenshots

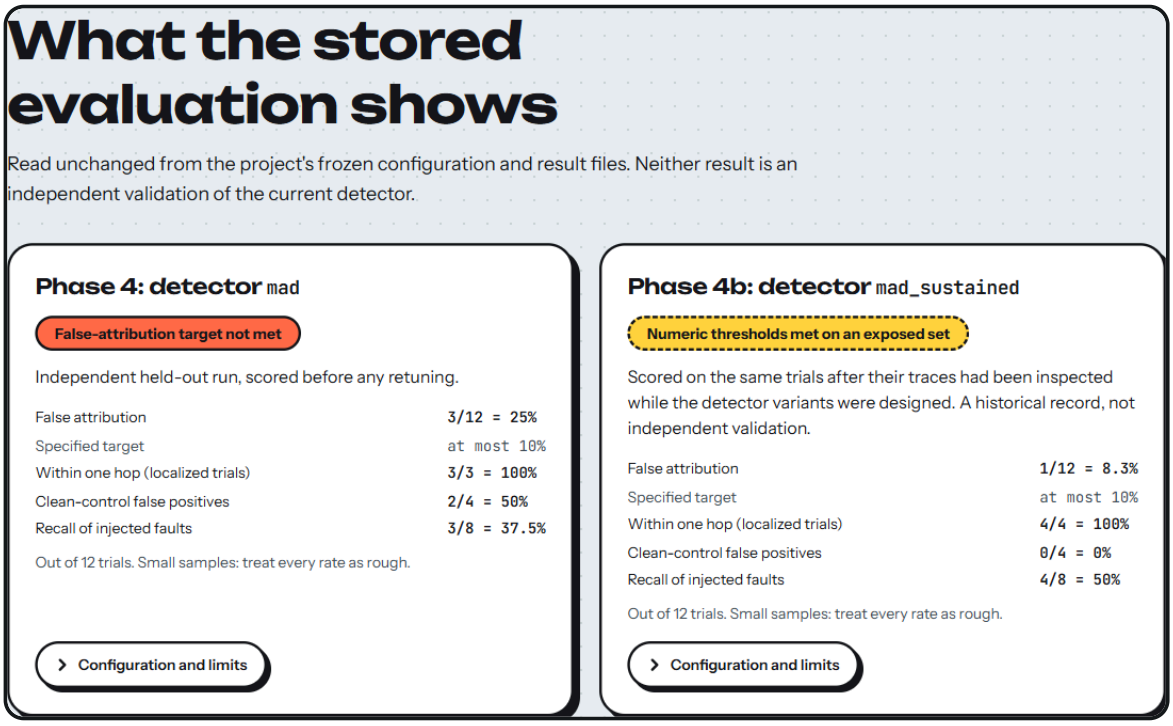


Figure D1. Stored evaluation panels: Phase 4 labelled "False-attribution target not met"; Phase 4b labelled "Numeric thresholds met on an exposed set" and "not independent validation".

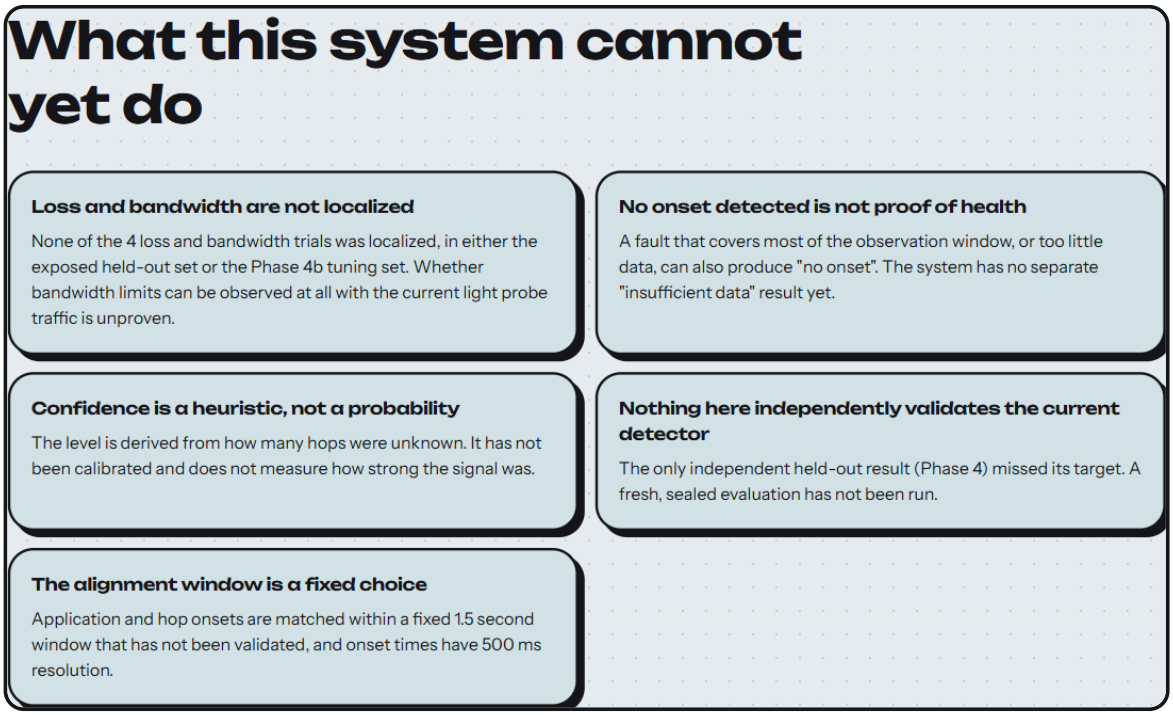


Figure D2. Limitations panel: loss and bandwidth not localized; no onset is not proof of health; confidence is a heuristic; nothing independently validates the current detector; the alignment window is a fixed choice.



Figure D3. Clean control (`p4-held-01-clean1`): "No significant degradation detected", with the caveat that no onset does not prove health.



Figure D4. `p4-held-12-unknown-plus-impair` : "Partially localized"; Hops 1 and 2 are unknown, "not assumed healthy".

E. How this report was produced

The report is generated by `submission/build/build_report.py` from hand-written HTML (`report_part*.html`) and a stylesheet; diagrams are drawn from the topology constants, measured-data figures by `make_figures.py` from the files in `experiments/` and `evaluate/` , and the PDF is printed with headless Chrome. Before printing, the build asserts every headline number against the result files. Screenshots were captured from the running production build and are stored unedited in `submission/screenshots/` .